

The Melting Point: A Time Series Exploration of Global Temperature Change

Taylor Kirk and Kamran Shirazi

University of San Diego

ADS 506: Time Series Modeling

December 8, 2025

Introduction

Climate change is one of the defining challenges of the modern era. While Earth's climate has always shifted over long periods, the rate of warming observed over the past century stands out as unusually fast. This rapid change is strongly connected to human activity. Industrial expansion increased the burning of fossil fuels, raised greenhouse gas concentrations, and contributed to rising global temperatures. These warmer temperatures affect sea levels, storm intensity, ecosystems, and many other environmental systems that people depend on.

In this project, we set out to better understand what is driving the warming trend and to explore whether time series forecasting methods can produce accurate predictions when supplied with the right explanatory variables. Our work focuses on data available through Berkeley Earth, an organization that collates temperature data from several sources to provide independent and open-source temperature records. Combining this data with several climate-related predictors such as greenhouse gas concentrations, ENSO events, volcanic activity, and solar irradiance was also explored. Through this approach we hope to identify which factors provide the strongest predictive power and what this means for understanding future warming. Our modeling includes baseline ETS, TSLM, and ARIMA approaches, followed by more advanced models that incorporate lagged and smoothed versions of climate drivers.

Problem Statement

The goal of this project is to understand which factors contribute most to rising global temperatures and to evaluate how much forecasting accuracy improves when meaningful climate predictors are included in time series models. Although CO₂ is known to drive long-term warming, short-term temperature fluctuations from El Niño/La Niña events, volcanic eruptions, and solar variation make it difficult to separate the underlying warming trend from natural variability.

Global monthly temperatures show a clear upward trend and strong seasonal patterns, yet common baseline models such as simple ETS or TSLM often fail to capture recent warming peaks or respond appropriately to major climate events. The gap, therefore, lies between these limited baseline models which overlook key physical drivers and a more realistic forecasting framework that integrates scientifically meaningful predictors to better represent both long-term warming and short-term variability.

The objectives of this project are to identify which predictors meaningfully improve forecasts, build models that account for anthropogenic warming and natural phenomena, and provide forecasts that individuals can interpret in terms of how human actions may influence future temperatures. Success means selecting a model that improves forecast accuracy relative to baseline approaches, produces residuals resembling white noise, and remains interpretable enough to understand how each predictor affects the results

Literature Review

Reliable global temperature datasets are essential for climate research. NASA's GISTEMP dataset provides monthly temperature anomalies dating back to the mid-19th century and combines land-surface air temperatures with ocean surface observations. These anomalies are calculated relative to the 1951–1980 baseline period, which is visible throughout the dataset used in our exploratory analysis. The GISTEMP methodology builds on foundational work by Hansen and Lebedeff (1987), who helped establish global temperature reconstruction techniques that continue to be refined. Their findings showed clear warming trends even when earlier studies mistakenly suggested global cooling. The methodology surrounding GISTEMP has continued to evolve, improving spatial coverage, adjusting for station biases, and refining sea surface temperature estimates.

Time series forecasting plays an important role in climate modeling. Because temperature data show trend, autocorrelation, and seasonal structure, models such as ETS and ARIMA are commonly used. ETS models work well when seasonal patterns remain stable, while ARIMA can capture both short-term and seasonal dependencies. Research from the National Center for Atmospheric Research suggests that global temperatures are well described by trend-driven or difference-stationary processes, which aligns with our KPSS findings.

However, univariate models often fall short because the climate system is influenced by more than internal statistical structure. Key drivers include greenhouse gases (CO_2 , CH_4 , N_2O), volcanic aerosols, ENSO cycles, and changes in solar irradiance. The IPCC's *Climate Change 2021: The Physical Science Basis* outlines how each of these factors influences global temperatures. ENSO events, for example, explain short-term temperature swings, while CO_2 concentrations influence long-term warming. This supports our modeling decisions to compare models with combinations of predictors, including lagged versions and smoothed ENSO indices.

The research literature also emphasizes the value of linking mechanistic climate drivers with statistical modeling. Our results reflect this. As shown in our report's later sections, adding predictors improves the performance of both TSLM and ARIMA models. In particular, the ARIMA model that included radiative forcing from CO₂, volcanic forcing, and a smoothed ENSO index performed well and produced white-noise residuals, suggesting that models combining physical science knowledge with statistical structure provide the clearest forecasting improvements.

Overall, the literature supports the approach we adopted: start with simple statistical models, diagnose their shortcomings, and incorporate relevant climate predictors to capture the underlying mechanisms driving observed temperature changes.

Explanation of Steps

The raw dataset was sourced from [Berkeley Earth](#) and contains 2,109 rows and 12 columns, including a year field, a numerical date field, the Monthly_Anomaly and its uncertainty, and multiple smoothed anomaly columns (annual, 5-year, 10-year, and 20-year means with corresponding uncertainties). The anomaly data comes in various forms from global land only estimates to as granular as the local and individual weather station level. For the purposes of this project it was decided to use the global estimates that merge land surface air temperature estimates with ocean sea surface water temperatures for a more comprehensive analysis. The observations are monthly and extend from 1850 to Sept 9, 2025.

Temperature data is often reported as anomalies, meaning as a difference from a defined baseline period. This is done to remove the noise present in absolute temperature measurements that result from differing elevations, urban v rural areas, large bodies of water etc. In this case, the baseline period for the raw data extends from Jan 1951 to Dec 1980.

Data Preparation and Cleaning

Minimal cleaning was required because the Berkeley Earth dataset is already well-processed and quality-controlled. The Month and Year fields were combined into a single Date column, and the data was converted into a tsibble for use with the Tidyverts framework.

A formal data quality review showed:

Missing values: No missing observations were present in the Monthly_Anomaly series or the merged absolute temperature series.

Outliers: No anomalous spikes or implausible temperature values were detected when visually inspecting the anomaly/temperature series or its decomposed components.

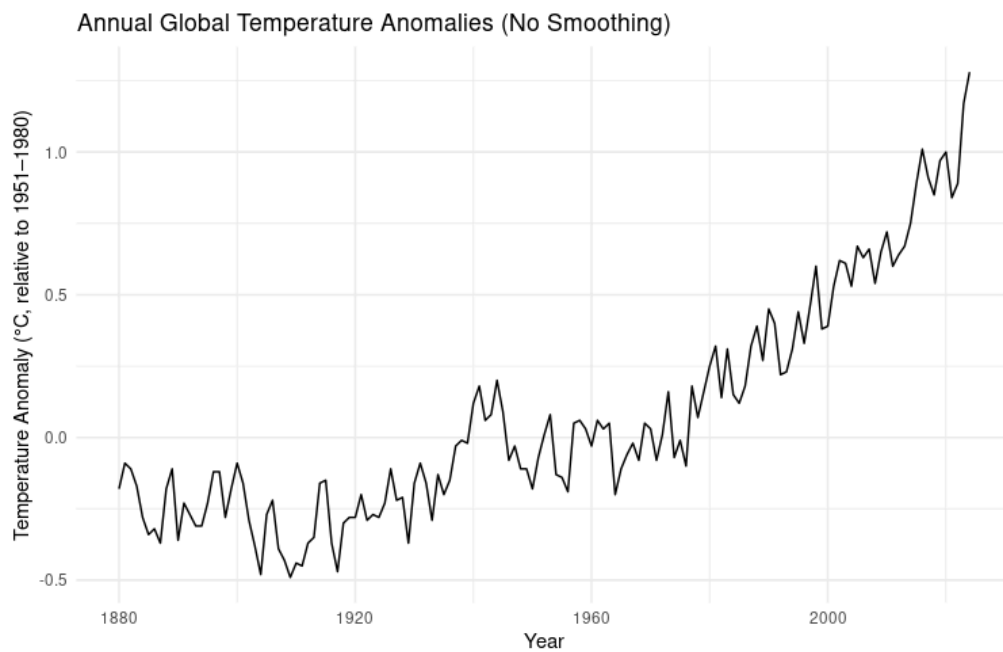
Consistency: All anomaly/temperature values followed expected ranges for global monthly averages, and date stamps were chronologically complete with no gaps.

Structure: All variables were typed correctly (e.g., numeric columns for anomaly measures, continuous monthly timestamps), allowing for immediate conversion into time-series formats once the Date column was created.

The anomaly series was initially used for exploratory analysis; however, it was converted to absolute temperatures after initial modeling. This addressed three limitations of anomalies for forecasting: varying baseline definitions across studies, removal of seasonality that was important to retain for model structure, and the increased interpretability of forecasts expressed in actual temperature units. Berkeley Earth's baseline monthly temperature vector was added to the anomaly series to construct a two column time series of absolute global temperatures.

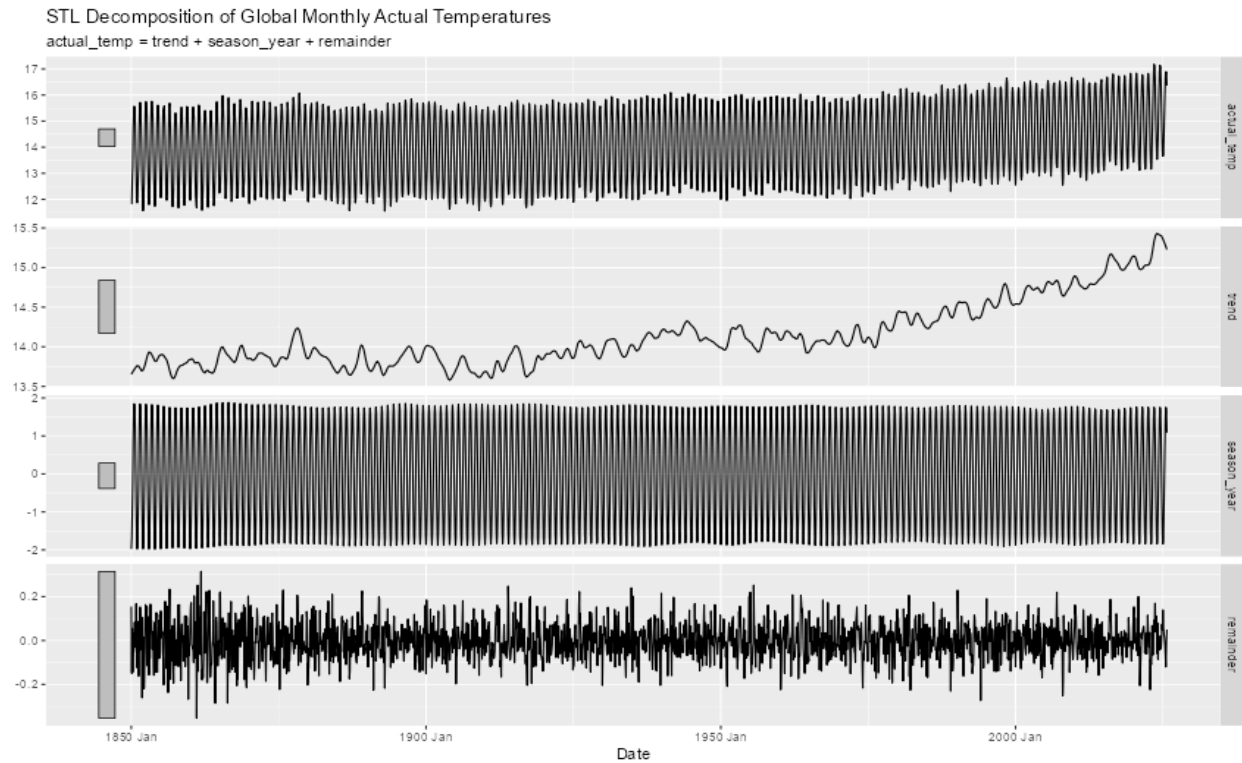
Exploratory Data Analysis (EDA)

One advantage of the anomaly data is that the trend in the time series is much more clear.

Figure 1*Annual Global Temperature Anomalies*

The anomaly data shows a trend that remains relatively stable and below 0°C from 1850 to about 1915. It then increases until roughly 1950, stabilizes through 1980 (the baseline period), and rises sharply and consistently thereafter.

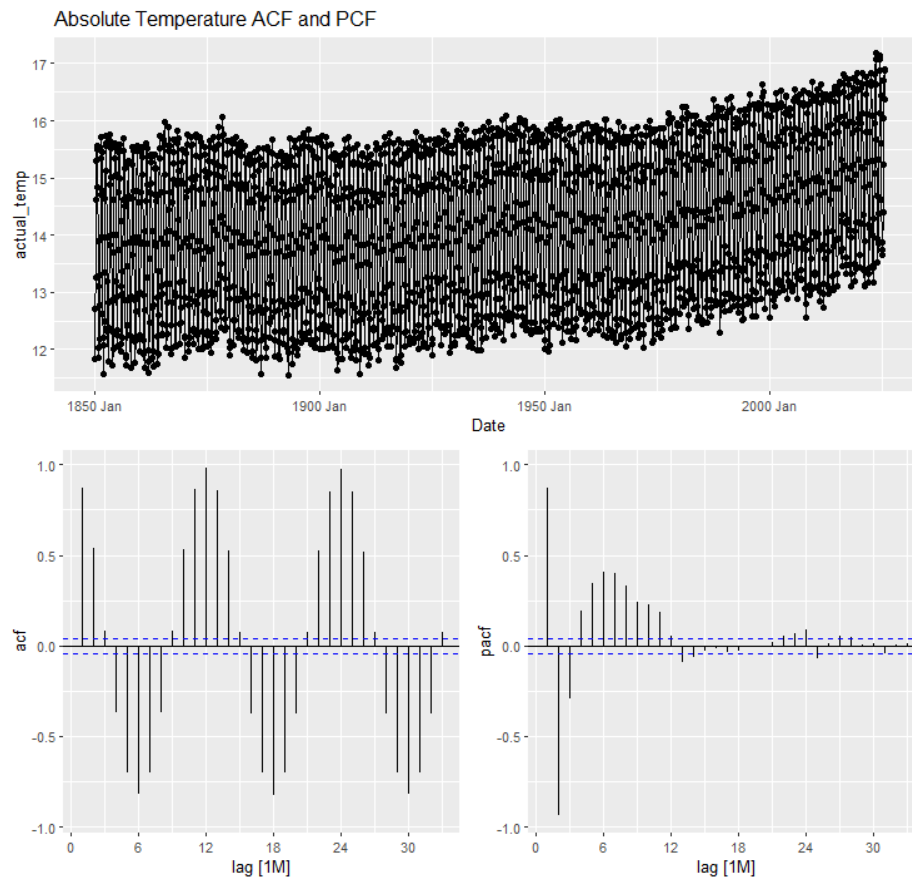
The temperature data components can be observed using a Seasonal and Trend decomposition using Loess (STL) model.

Figure 2*Temperature Time Series Decomposition*

The seasonal component in the absolute temperature data is strong and dominant. It also appears to compress over time. This could be the result of more precise temperature measurements, or perhaps the warming trend is more prevalent for the winter compared to the summer months. The remainder is the smallest component showing that temperatures are mostly explained by seasonal variation and the changing trend.

Autocorrelation

Other notable findings from an initial analysis of the data is the presence of significant autocorrelation.

Figure 3*Absolute Temperature ACF and PCF*

Seasonal autocorrelation is immediately apparent, with clear spikes at seasonal lags. This aligns with the expected annual temperature cycle and supports the need for seasonal differencing when modeling.

Stationarity and Transformation Checks

KPSS Test: $p < .01 \rightarrow$ non-stationary

unitroot_ndiffs(): One non-seasonal difference required

unitroot_nsdiffs(): One seasonal difference required for the raw temperature series

Guerrero lambda: $\lambda \approx 1.5 \rightarrow$ suggests mild left-skew; transformation may improve stabilization

Modeling Steps

Modeling Pipeline

Across this analysis, three model families were evaluated: Error–Trend–Seasonality (ETS), Time Series Linear Models (TSLM), and AutoRegressive Integrated Moving Average (ARIMA).

The initial step was fitting an auto-selected ETS model to the anomaly series. The training set included all observations before January 2020, and the validation period covered January 2020 through the most recent observation—yielding a 5-year, 9-month forecast window.

Next, all three model types were applied to the full temperature series without predictors. ETS and TSLM models were trained using time-series cross-validation: the initial training window covered 1850–1923 (125 years), and eleven 5-year rolling test splits were generated. Because auto-selected ARIMA models require substantially more computation, they were first evaluated on a single 5-year holdout split, using all data prior to September 2020 for training and the following five years for validation. The strongest models from each family were then compared using the cross-validation framework.

TSLM and ARIMA models were subsequently trained with external predictors. Given the number of engineered monthly observations in the predictor data and the fact that the warming trend of interest does not begin in earnest until more recent decades, it was decided to use the data from 1958 until present. TSLM models again used rolling-origin cross-validation: the initial training window spanned 1958–September 2005 (48 years), followed by five 5-year rolling test splits. ARIMA models, due to runtime constraints, were initially assessed on a single holdout split with training data from 1958–September 2020 and a 5-year validation period thereafter.

Finally, the best-performing models were refitted using the post-1957 cross-validated training structure for a final comparison.

Metrics Used

Root Mean Squared Error (RMSE) was the primary metric used to evaluate forecasting performance on the validation data. RMSE measures the average magnitude of the model’s errors, with a stronger penalty applied to larger errors due to the squaring of residuals. This made it the most relevant accuracy measure for assessing how well each model would generalize to unseen data.

When models showed similar RMSE values, and the same transformation for the response variable was used, the Akaike Information Criterion (AIC) was used to compare them. AIC evaluates goodness of fit while applying a penalty for model complexity, which is especially important when external predictors are included.

Mean Absolute Error (MAE) was also examined as an additional measure of average error magnitude. Unlike RMSE, MAE weights all errors equally and therefore provides a complementary view of model performance.

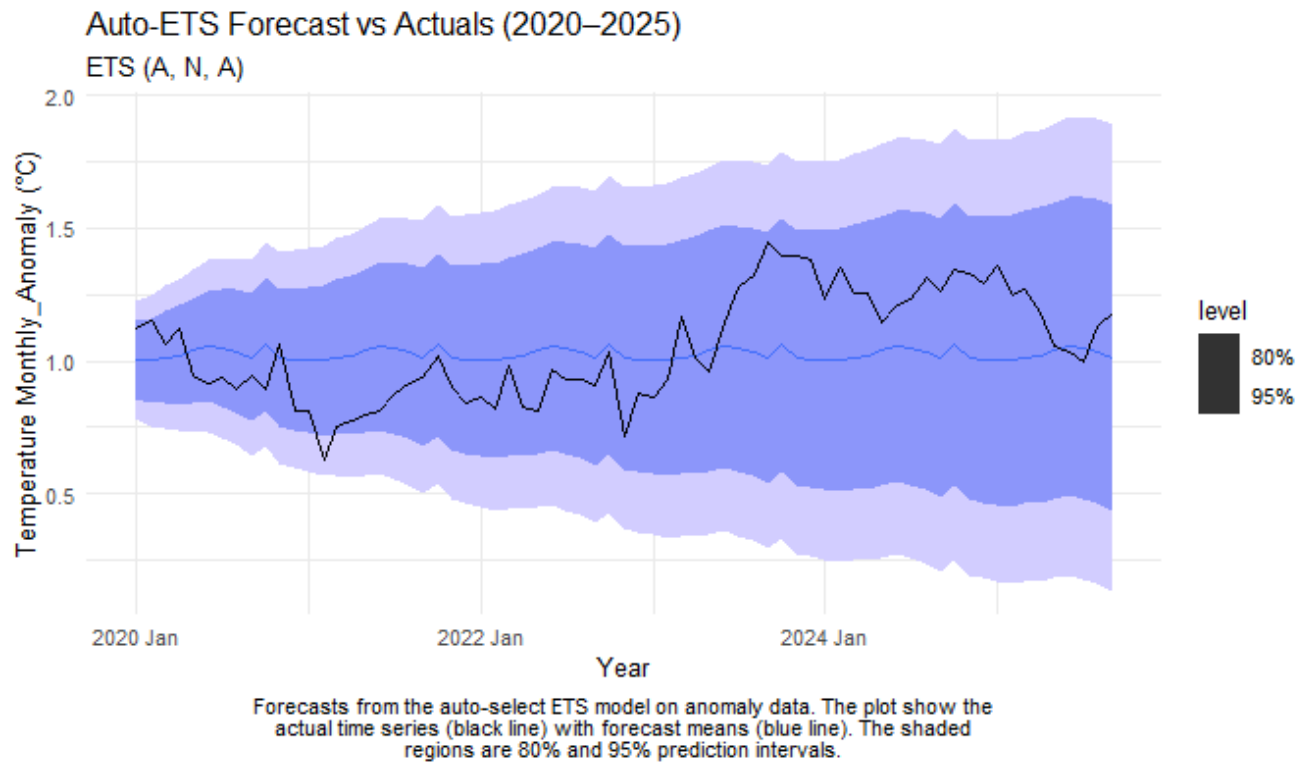
Mean Error (ME) was used to assess the direction and extent of bias in the forecasts. Positive or negative ME values indicate systematic under- or over-forecasting, while values near zero suggest little to no overall bias.

Mean Absolute Percentage Error (MAPE) was not used, as percentage-based metrics are not informative for temperature data. Temperature values can cross zero or vary within narrow ranges that make percentage errors misleading.

Autocorrelation was evaluated both visually, using ACF and PACF plots, and statistically, using the Ljung–Box test. A Ljung–Box test holds the null hypothesis that the residuals come from a white-noise series, so a low p -value means that the null hypothesis can be rejected. These diagnostics were used to assess whether residual patterns remained in the fitted models and to compare the adequacy of different specifications.

ETS Modeling of Anomaly Data

The algorithm selected an ETS(A, N, A) model, meaning it uses an additive error, no trend component, and an additive seasonal component. The estimated smoothing parameters were $\alpha = .4671$ and $\gamma = .0001$. The level smoothing parameter α controls how quickly the model updates the underlying level of the series, while the seasonal smoothing parameter γ governs how quickly the seasonal pattern adapts to changes over time. Higher values indicate the model is giving more weight to recent observations, whereas very low values suggest that the corresponding component is effectively stable. The resulting ETS forecast is shown below.

Figure 4*Auto-ETS Forecast vs Actuals (2020-2025)***ETS Modeling of Temperature Data**

Seven different ETS models were fit to the cross-validated data. Below are the ranges of the validation metrics and smoothing parameters among all the models.

Table 1*Validation Metrics and Smoothing Parameters*

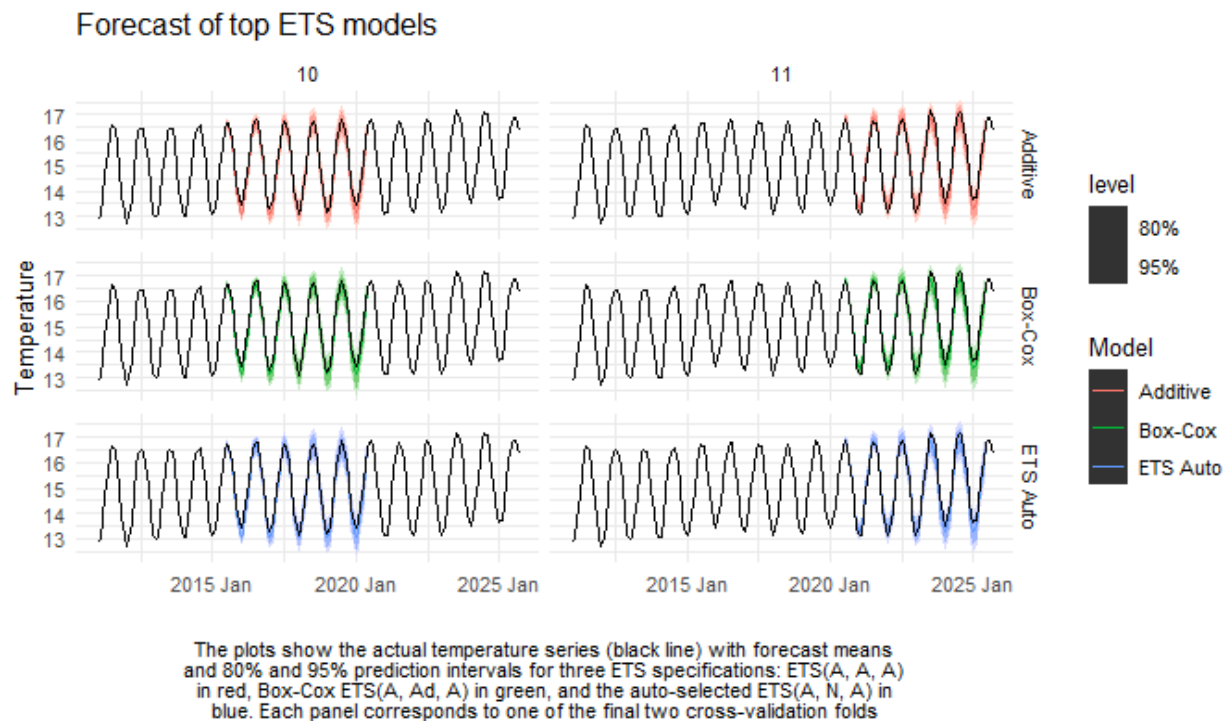
Metrics				Parameters				
RMSE	ME	MAE	Ljung p-value	AIC	α	β	γ	ϕ
0.1724 - 0.1919	-0.0179 - 0.0261	0.1409 - 0.1627	0 - .0041	-3443 - 10079	.2812 - .4469	.0001 - .0057	.0001 - .0755	~ .98

The best-performing models were additive ETS variants. The top model selected was ETS(A, A, A), and performance was similar regardless of whether the target variable was transformed or whether the trend component was damped. Multiplicative models consistently showed poorer performance, corresponding to the relatively gradual long-term changes and stable seasonal pattern present in the data.

The level smoothing parameters α ranged from .2812 to .4469, indicating moderate updating of the level over time. The trend smoothing parameters β were near zero in all models that included a trend component, reflecting minimal trend updating. The seasonal smoothing parameters γ were also close to zero, consistent with a largely stable seasonal pattern. For models that incorporated damped trends, the damping parameter ϕ was approximately .98, indicating only slight damping when applied.

Figure 5

Forecast of Top ETS models



Visualizing the top two models and the worst performing model on the two most recent splits in the cross-validation data, there is wider uncertainty at the peaks, and some underprediction going on there

as well. The models seem to do well with the fall and spring months but struggle more in the months with more extreme temperatures.

The wide range of AIC values across the ETS models reflects differences in the scale of the response under the various transformations. Log transformations produce smaller numerical values, which increase the log-likelihood and therefore lower the AIC, while Box–Cox transformations have the opposite effect. Because these shifts are driven by scale rather than true differences in model fit, RMSE is the more appropriate metric for comparing performance across models with different transformations, and AIC should be interpreted only when comparing models within the same transformation group.

As can be seen in the Ljung-Box test column, no model has a p -value over .0041 indicating that autocorrelation is most likely present in all the residuals.

External Predictors

Global temperatures are affected by a multitude of factors. The various external predictors used in TSLM and ARIMA modeling and why will be listed below, then the findings from TSLM and ARIMA modeling will be discussed.

Total Solar Irradiance (TSI)

TSI is a measure of energy that the Earth receives from the Sun across all lengths of electromagnetic waves and plays a small but meaningful role in global temperatures. The measurement is taken at the top of Earth's atmosphere and is reported in Watts per square meter. The data from this predictor comes from [Total Solar Irradiance CDR | National Centers for Environmental Information \(NCEI\)](#) and is available in monthly observations dating back to 1874.

ENSO Index

ENSO is an acronym for El Niño - Southern Oscillation. The index is the measurement of the difference in sea surface temperatures (SST) from the long-term average (long-term defined as the period between 1991 and 2000) in the equatorial Pacific Ocean (Bureau of Meteorology, n.d.). These weather events play a significant role in the global climate. Part of the challenge of sourcing this data is that there are several different indices reported from several different locations around the world. The mostly commonly used one comes from the Niño 3.4 region. For this project the Ensemble Oceanic Niño Index

(ENS ONI) is used and can be read about here [Ensemble Oceanic Niño Index](#). In addition to using the numerical index, dummy variables were created to indicate the presence of an El Niño or La Niña weather event for the relevant time period. A more thorough explanation of the index and the engineering steps taken can be found in the appendix.

Carbon Dioxide (CO₂) Concentration

It's well known that Greenhouse Gases (GHG's) are the main drivers of global warming, with CO₂ making up the majority of emissions. The most reliable CO₂ concentration observations are reported in parts per million (ppm) and are recorded at the [Mauna Loa Observatory](#). The Mauna Loa Observatory did not begin recording CO₂ concentration data until 1958, so CO₂ ice core data was sourced from [here](#) to engineer monthly data back to 1850. A thorough explanation of those steps and other feature engineering performed can be found in the appendix.

Methane (CH₄) Concentration

CH₄ is another one of the notorious GHGs affecting the environment. Similar to CO₂, the most reliable estimates come from the Mauna Loa Observatory, albeit measured in parts per billion (ppb). Reliable monthly estimates were not available until 1977 and the same process used for CO₂ was applied to engineer monthly estimates from the annual estimates.

Nitrous Oxide (N₂O) Concentration

Another major component of GHG's, the same explanation from the CO₂ and CH₄ predictors applies here.

Radiative Forcing

Radiative forcing is not a measurement but rather a scientific principle. It is used as a way to quantify how the Earth's balance of energy (in this case measured as Watts per meter squared) changes when another variable changes. For this project, the radiative forcing of CO₂ was measured. It was decided to only use CO₂ as this variable is recognized as having the largest forcing effect of the various physical drivers of temperature change. The equation used to calculate the radiative forcing variable is as follows

$$RF = 5.25 \times \log(C_t / C_0)$$

Where C_0 is defined as the reference CO_2 concentration level prior to the industrial revolution. 278 ppm is the most widely used value.

Volcanic Activity

Volcanic eruptions can have profound effects on the climate that last anywhere from 1 to 3 years. The National Oceanic and Atmospheric Administration (NOAA) keeps a record of all known volcanic eruptions dating back 1000's of years and can be found [here](#). Two variables to indicate volcanic activity were engineered from this data. The steps taken to engineer this data and the data was filtered can be found in the appendix.

After adding in the above external predictors and the engineered versions, the dataset had a total of 28 columns and was ready for TSLM and ARIMA modeling.

TSLM Modeling of Temperature Data

Seven TSLM models were fit without external predictors, and a total of 20 models were fit with external predictors. The metric ranges for each subset of modeling are below.

Table 2

TSLM Modeling Metrics

	RMSE	ME	MAE	Ljung p -value	AIC
No predictors	0.3198 - 1.4553	0.2660 - 0.6108	0.2881 - 1.2584	0	-14919- 1031
With predictors	0.1183 - 0.1766	-0.0063- 0.1017	0.0927- 0.1463	0	-5721 - -731

Adding predictors to the TSLM models resulted in substantial improvements in several performance metrics. Among models without predictors, the ME values indicated a consistent tendency toward under-forecasting, whereas models that included predictors showed little to no systematic bias. The AIC values were more compressed across the predictor-enhanced models; however, AIC reflects in-sample fit

and assumes independence among predictors. In contrast, the models with predictors produced notably better results on the validation set, despite some remaining autocorrelation in the residuals.

Based on the reported metrics, the best-performing TSLM specification applied a Box–Cox transformation of the response variable ($\lambda = 1.5$), incorporated seasonal effects, and included both a 10-year-lagged CO₂ concentration and a 12-month smoothed ENSO index as predictors. Model performance improved further when the trend term was excluded.

Overall, the strongest TSLM models typically applied a Box-Cox transformation and included an ENSO-related predictor and a lagged CO₂ predictor, while other candidate variables contributed little additional improvement.

ARIMA Modeling of Temperature Data

Seven ARIMA models were fitted without predictors, then 27 models were trained with predictors. Initially, the single training/test split was used to ascertain performance using various parameters and predictors. Then the most promising models were training on the cross-validated data. Metric ranges are found below.

Table 3

ARIMA Modeling metrics

	RMSE	ME	MAE	Ljung p -value	AIC
No predictors	0.2174 - 0.2929	-0.1230 - 0.2035	0.1714 - 0.2365	0 - 0.6324	-13046 - 2933
With predictors	0.1183 - 0.3086	-0.0304 - 0.2180	0.0927 - 0.2519	0 - 0.851	-5118 - 865

For the ARIMA models without predictors, the auto-selection procedure consistently returned specifications of the form **ARIMA(1,0,0)(0,1,1)** with drift. These models showed positive ME values and exhibited residual autocorrelation at seasonal and biennial lags. Because non-seasonal differencing was not included in the auto-selected models, additional specifications were fitted by enforcing a first difference and by evaluating a custom structure, **ARIMA(3,1,2)(1,1,2)** without a constant. This custom

model produced the strongest results among the models with no predictors, with the lowest RMSE and MAE, minimal bias, and no significant residual autocorrelation.

When external predictors were added, a broader set of specifications performed well. Log transforming the response variable consistently improved accuracy, and models including ENSO related predictors showed further gains and reductions in residual autocorrelation. CO₂ related predictors generally improved model fit, while N₂O and CH₄ contributed little additional value. Across well performing models, seasonal AR and MA components were the most influential structural terms.

The top performing model combined radiative forcing from CO₂, a volcanic activity variable with exponential decay, and a three-month smoothed ENSO index, with the specification **ARIMA(2,0,2)(0,1,2)**. Several models produced similar validation metrics and AIC values, and all exhibited negligible residual autocorrelation.

Best Non-Predictor Model Comparison

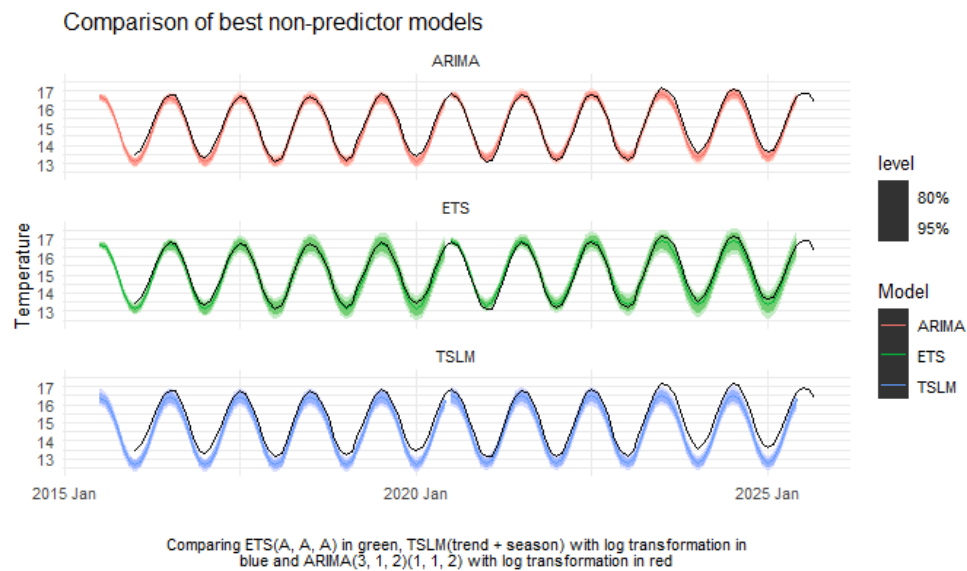
The following non-predictor models were selected for comparison:

- **ETS(A, A, A)**
- **TSLM(trend + season)**
- **ARIMA(3, 1, 2)(1, 1, 2)** without a constant

The performance metrics and forecast visualizations are shown below.

Table 4*Error Comparison of Best Non-Predictor Models*

Model	RMSE	MAE	ME
ARIMA	0.1634	0.1323	0.0482
ETS	0.1748	0.1429	-0.0004
TSLM	0.3191	0.2880	0.2637

Figure 6*Comparison of Best Non-Predictor Models*

Across these models, ARIMA and ETS produced lower errors than TSLM, and both showed minimal bias based on ME values. The ETS model displayed wider forecast intervals, while the ARIMA model was the only one with no significant residual autocorrelation. One observation is that, across the 11 cross-validation splits, the ARIMA model returned a NULL fit for one split, indicating some potential sensitivity in that specification.

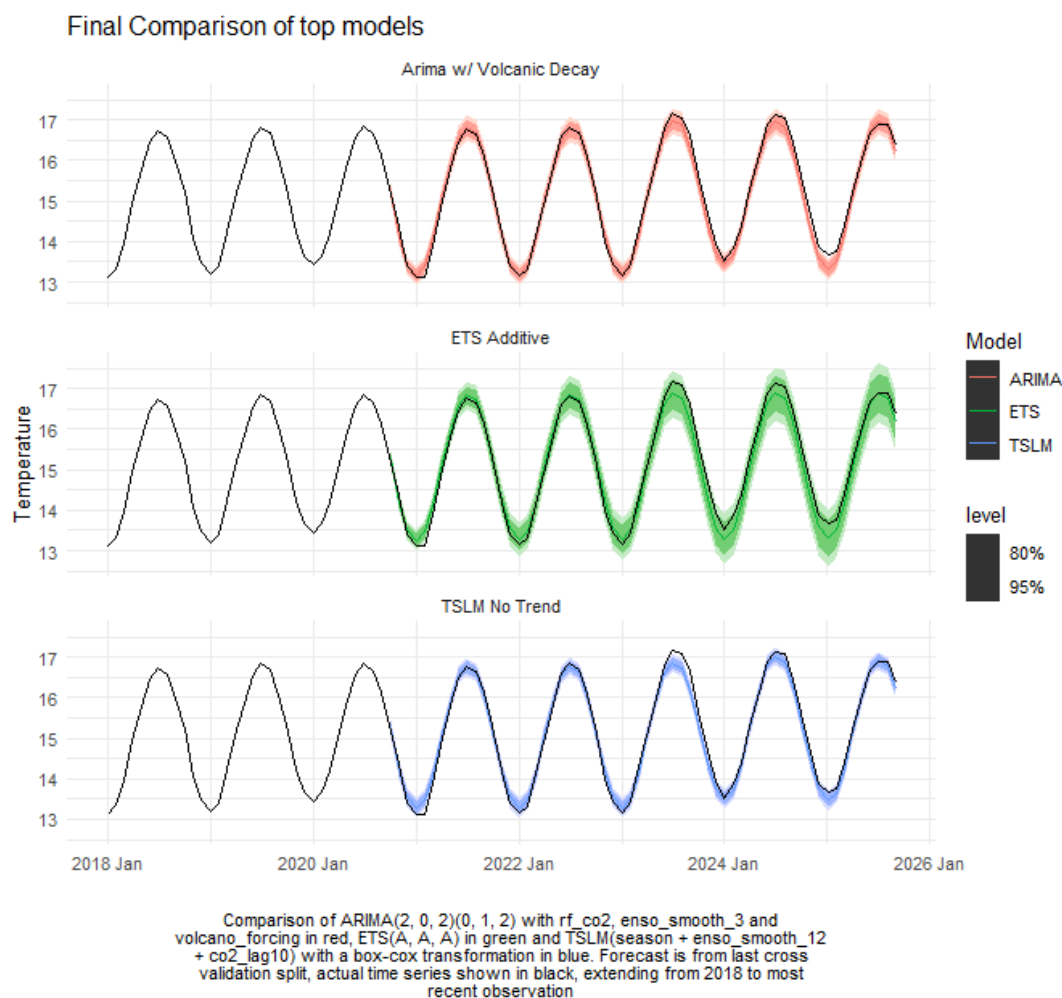
Best Overall Model Comparison

Three ETS models, 3 TSLM models and 4 ARIMA models for a total of 10 models were chosen for a final comparison on the post-1957 cross-validated training data. Models compared and results are below.

Table 5

Final Model Comparison

Model Family	Model	Trans	ENSO	Lagged CO ₂	Radiative CO ₂	Volcanic Decay	RMSE	MAE	ME	AIC	Ljung–Box
TSLM	TSLM (season)	Box–Cox $\lambda = 1.5$	✓ (12-mo)	✓ (10-yr)	–	–	0.1183	0.0927	–0.0032	–955	0
	TSLM (season)	Box–Cox $\lambda = 1.5$	✓ (12-mo)	✓ (10-yr)	–	✓	0.1191	0.0936	0.0013	–964	0
	TSLM (trend + season)	Box–Cox $\lambda = 1.5$	✓ (6-mo)	✓ (10-yr)	–	–	0.1217	0.0950	–0.0063	–947	0
ARIMA	ARIMA(2,0,2)(0,1,2)	Log	✓ (3-mo)	–	✓	✓	0.1233	0.0946	0.0005	–4300	.851
	ARIMA(2,0,2)(0,1,2)	Log	✓ (3-mo)	–	✓	–	0.1304	0.1012	–0.0011	–4271	.679
	ARIMA(1,0,1)(0,1,2)	Log	✓	–	–	–	0.1356	0.1083	–0.0048	–4273	.513
	ARIMA(1,0,1)(0,1,2)	Log	✓	✓ (10-yr)	–	–	0.1358	0.1076	–0.0304	–4261	.429
ETS	ETS(A,A,A)	None	–	–	–	–	0.1569	0.1272	0.0397	1293	.0441
	ETS(A,A,A)	Box–Cox $\lambda = 1.5$	–	–	–	–	0.1621	0.1331	0.0342	2914	.0878
	ETS(A,Ad,A)	None	–	–	–	–	0.1656	0.1348	0.0604	1250	.0769

Figure 7*Final Comparison of Top Models*

Across all models evaluated, the ARIMA, TSLM, and ETS families showed varying strengths depending on the chosen predictors and specifications. The TSLM models produced slightly lower error metrics overall but consistently exhibited significant residual autocorrelation and higher AIC values compared with the ARIMA models. The ETS models showed weaker performance on error metrics and also retained notable autocorrelation in their residuals.

Among the ARIMA models, several specifications performed similarly on validation metrics. The two highest-performing models incorporated radiative forcing and volcanic activity as predictors. Another strong model was **ARIMA(1, 0, 1)(0, 1, 2)** with ENSO index and 10-year-lagged CO₂ concentration as

predictors, which demonstrated competitive error metrics, acceptable AIC values, and reduced residual autocorrelation relative to other models in this group.

Given the set of candidate models, the **ARIMA(1, 0, 1)(0, 1, 2)** model with ENSO and lagged CO₂ predictors was selected for scenario-based forecasting due to its strong overall performance and the feasibility of generating values for its external regressors in subsequent forecasting exercises.

Scenario Forecasting

A scenario-forecasting framework was developed to generate long-range temperature projections under alternative assumptions about external predictors. The temperature forecasting model requires two exogenous inputs: **atmospheric CO₂ concentration** and the **ENSO index**. Because short-term ENSO fluctuations are irregular and not forecastable using standard time-series methods, long-range ENSO behavior was represented through simulated sequences. Three ENSO scenarios were constructed by repeating historical 50-year patterns:

- a sequence representing average El Niño conditions,
- a sequence with relatively warmer El Niño conditions, and
- a sequence with relatively cooler El Niño conditions.

CO₂ Concentration Modeling

To forecast atmospheric CO₂ concentration, several models were evaluated, and the selected specification was an **ARIMA(1, 1, 1)(0, 1, 1)** seasonal model. The model used two external regressors:

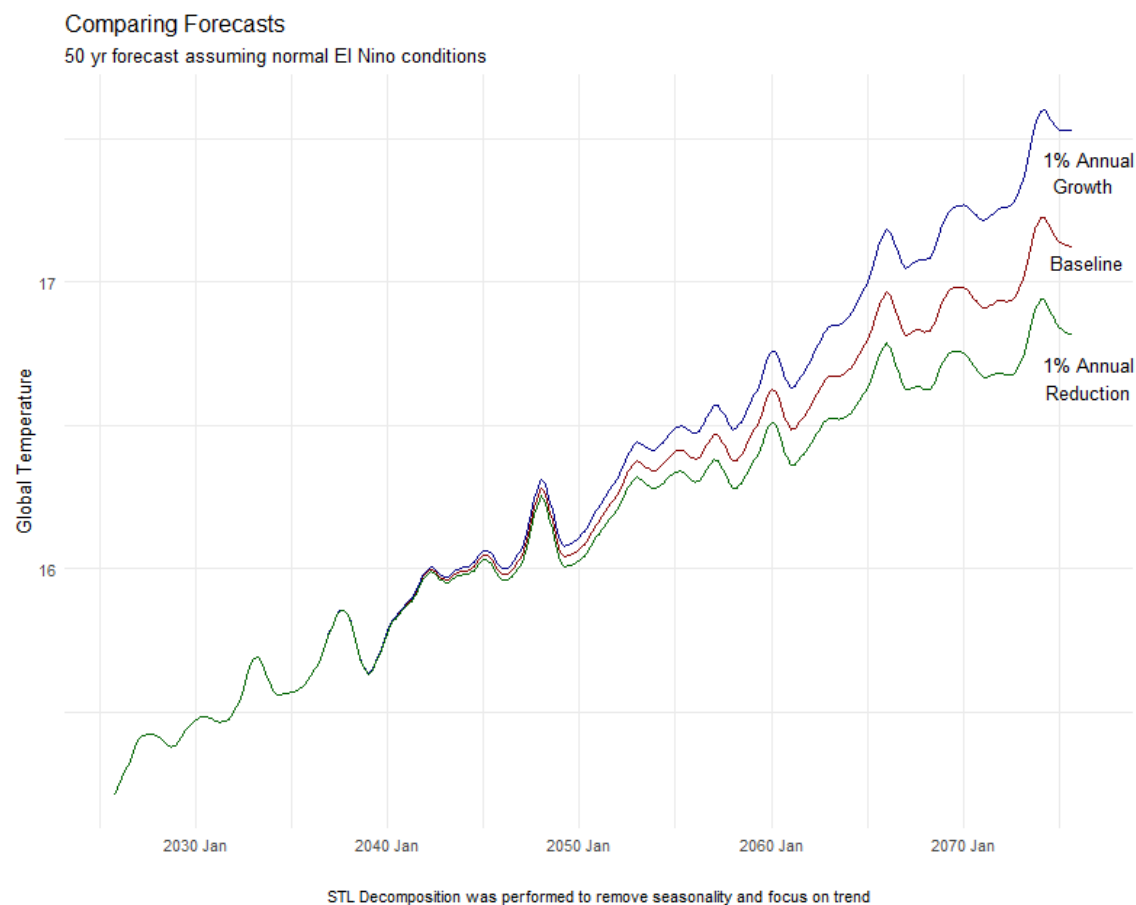
- Aggregate CO₂ emissions, constructed as the cumulative sum of annual emissions beginning in 1850, converted to a monthly series and lagged by 12 months.
- A 12-month smoothed ENSO index, lagged by four months.

Emissions were obtained from [EDGAR - The Emissions Database for Global Atmospheric Research](#) and [Greenhouse gas emissions - Our World in Data](#).

Forecasting Procedure

Scenario forecasts were generated by specifying an annual percentage change in emissions and choosing a forecast horizon. The CO₂ model produced mean forecasts of CO₂ concentration for each ENSO scenario. These predicted CO₂ paths were then supplied to the selected temperature model to produce long-range temperature forecasts.

The visual below displays a comparison of three CO₂ scenarios – baseline emissions (equal to the aggregate level at the end of 2024), 1% annual compounded growth, and 1% annual compounded reduction – projected over 50 years under average El Niño conditions.

Figure 8*Comparing Forecasts***Discussion****ETS and Training Data**

The initial ETS models trained on the anomaly data highlighted the limitations of using anomalies as the response variable. With seasonal variation removed, the models effectively reduced to predicting the average level of the series, producing nearly flat forecasts and wide prediction intervals. Fitting ETS models to the full temperature series immediately improved stability and accuracy.

However, the ETS models trained on the full historical record suggested that using all available data was not ideal for capturing modern warming. The small β smoothing parameters indicated minimal trend updating, and small α parameters showed a strong reliance on earlier historical temperatures, periods

characterized by relatively stable year-to-year variation. This was consistent with the higher ME values seen in non-predictor TSLM and ARIMA models, all of which under-forecasted temperatures. Together, these results suggested that the long period of historically stable temperatures was obscuring the trend the models needed to learn.

To better represent the warming regime and make use of available predictors, a 1958 training cutoff was adopted, corresponding to the beginning of reliable monthly CO₂ concentration estimates.

TSLM Findings

Adding predictors substantially improved the performance of TSLM models. Models without predictors showed clear under-forecasting based on their ME values, whereas adding ENSO and CO₂ predictors reduced this bias and improved all major accuracy metrics. AIC values also became more comparable across models once predictors were included, though AIC remains limited by assumptions of predictor independence.

The strongest TSLM model used a Box-Cox transformation ($\lambda = 1.5$), seasonal terms, a 10-year lagged CO₂ predictor, and a 12-month smoothed ENSO index. Notably, excluding the explicit trend term improved performance, suggesting that the predictors themselves were capturing the relevant structure. Across all TSLM models, the combination of a transformation, ENSO information, and lagged CO₂ consistently yielded the best results, while other predictors made little difference.

ARIMA Findings

For the models without predictors, auto-selection consistently chose **ARIMA(1,0,0)(0,1,1)** with drift, even though stationarity tests suggested non-seasonal differencing was appropriate. These models tended to under-forecast and showed persistent autocorrelation at seasonal and biennial lags. Adding a forced non-seasonal difference and fitting a custom ARIMA(3,1,2)(1,1,2) model eliminated residual autocorrelation and produced the best results among models without predictors.

With predictors, ARIMA models displayed a wider range of behaviors. Several consistent patterns emerged:

- Log transformation of the response variable produced the largest overall improvements, often reducing RMSE by 50% or more.

- ENSO-related predictors consistently improved forecasting accuracy and reduced residual autocorrelation.
- CO₂-related predictors were the next most influential.
- N₂O and CH₄ predictors offered little benefit and occasionally degraded performance.
- Seasonal MA and AR components were the most important parameters structurally.

The best-performing ARIMA models incorporated radiative forcing, volcanic activity, and a 3-month smoothed ENSO index, with **ARIMA(2,0,2)(0,1,2)** performing especially well. However, several top models produced nearly identical metrics and diagnostics, suggesting their relative ranking would likely vary with modest changes in the predictor set or time window.

Final Decision

The goals of the project were to model global temperatures accurately, identify meaningful predictors of the warming trend, and support scenario forecasting based on changes in emissions or ENSO conditions. Although TSLM models produced slightly lower error metrics, they exhibited substantial residual autocorrelation. ETS models performed less well overall and showed similar issues.

Among ARIMA models, the best metrics were associated with models using radiative forcing and volcanic activity; however, both predictors present challenges for scenario forecasting. Radiative forcing was modeled using a simplified approximation of the IPCC formula, and volcanic activity is inherently unpredictable. In contrast, CO₂ concentration can be forecasted reliably, and ENSO scenarios can be simulated using historical patterns.

For these reasons, the **ARIMA(1,0,1)(0,1,2)** model with **10-year lagged CO₂** and **ENSO** as predictors was selected as the final model. While not the single best model in terms of RMSE alone, it balanced accuracy, interpretability, predictive stability, and feasibility for long-range scenario generation.

Next Steps

Due to the limited nature of this project, there are several extensions to this work that should be looked into.

Impact of Renewable Energy

This project explored global aggregate CO₂ emissions and their effect on CO₂ concentration. Next steps would be exploring the effect of changing energy sources would have on CO₂ concentration and by extension, global temperatures. What impact does changing energy consumption by 10% today to solar energy from coal power plant have on CO₂ concentration in 20 years? Which renewable energy source (solar, wind, nuclear etc) leads to the largest reduction in CO₂ emissions on a per unit basis? These are questions that could be answered with a more thorough analysis of specific energy sources.

Complex Models

Global temperatures are affected by 100's of factors and simpler models such as those explored here might be prone to overfitting if too many are used. More complex models such as XGBoost would be worth exploring to see if they can improve forecasts and reduce uncertainty.

Improving Regressor Accuracy

There are other techniques that could improve the quality of regressor data used in this project. Radiative forcing for example showed promise, and using the IPCC standard radiative forcing calculation for CO₂ could improve results. In addition, a very simple estimate was used to engineer the effects of volcanic activity, but more precise techniques that could assess the impacts of each individual event exist and could be used to improve the quality of forecasts of models using that as a predictor.

Land Use v Consumption

In this project, total global emissions were used to calculate aggregate emissions and forecast CO₂ concentration. However, there is a recognized difference between CO₂ emissions emitted through consumption (such as electricity use) and CO₂ emissions emitted by changes in how land is used (such as clearing forests for agriculture). CO₂ emissions due to changes in land use are much less precise, and colloquially, when individuals think of reducing carbon emissions by a certain amount, they are typically thinking about carbon emissions from consumption. By separating these emission categories, forecasts could become clearer and accuracy might improve.

Conclusion

Global temperatures continue to rise, and the challenge addressed in this project was to determine which factors most strongly shape that trend and how incorporating scientifically meaningful predictors can improve temperature forecasting. This project confirmed that relying solely on simple baseline

models struggle to capture recent warming peaks and fail to account for the physical drivers of both long-term and short-term temperature variation.

By integrating predictors such as CO₂ concentration and ENSO weather patterns, the models were able to isolate the underlying warming signal more effectively while still responding to natural variability. Across all the approaches evaluated, the most balanced and practical choice for long-range forecasting was the **ARIMA(1,0,1)(0,1,2)** model with lagged CO₂ and ENSO as external regressors. Although some alternative ARIMA specifications achieved slightly lower error metrics, this model provided strong accuracy, interpretable structure, and feasible inputs for scenario generation making it the most suitable for the project's goals.

The key takeaway is that meaningful improvements in forecast performance come from aligning time-series structure with the physical processes that drive global temperatures. When models incorporate variables that reflect actual climate mechanisms, the results become not only more accurate but also more actionable. This clarity matters for people and policymakers looking to better understand how human activities influence future warming and how different choices today shape the temperature trajectories of tomorrow.

References

Berkeley Earth. (2025). *High-resolution monthly global average temperature* [Data file]. Retrieved from https://storage.googleapis.com/berkeley-earth-temperature-hr/global/Global_TAVG_monthly.txt

Bureau of Meteorology. (n.d.). *About the El Niño/Southern Oscillation (ENSO) Indices*. <http://www.bom.gov.au/climate/enso/indices/about.html>

Coddington, O., Lean, J. L., Lindholm, C., & Pilewskie, P. (2024). *NOAA Climate Data Record (CDR) of NASA NOAA LASP Total Solar Irradiance (NNLSI), Version 3* [Data set]. NOAA National Centers for Environmental Information. <https://doi.org/10.25921/k2ff-p712>

Hansen, J. E., & Lebedeff, S. (1987). *Global trends of measured surface air temperature*. *Journal of Geophysical Research: Atmospheres*, 92(D11), 13345–13372. <https://doi.org/10.1029/JD092iD11p13345>

Intergovernmental Panel on Climate Change. (2021). *Climate Change 2021: The Physical Science Basis*.
<https://www.ipcc.ch/report/ar6/wg1/>

NASA Goddard Institute for Space Studies. (n.d.). *GISTEMP surface temperature analysis (GISTEMP v4): Global surface temperature anomaly data*. <https://data.giss.nasa.gov/gistemp/>

NASA Goddard Institute for Space Studies. (2025, April 3). *Background of the GISTEMP analysis*.
<https://data.giss.nasa.gov/gistemp/history/background.html>

National Center for Atmospheric Research. (n.d.). *Global surface temperature data (GISTEMP): NASA Goddard Institute for Space Studies (GISS)*.
<https://climatedataguide.ucar.edu/climate-data/global-surface-temperature-data-gistemp-nasa-goddard-institute-space-studies-giss>

National Centers for Environmental Information. (n.d.). *Significant Volcanic Eruption Database* [Data set].
 NOAA National Centers for Environmental Information. <https://doi.org/10.7289/V5JW8BSH>

NOAA Global Monitoring Laboratory. (2025, December 5). *Trends in CO₂, CH₄, N₂O, SF₆*. U.S. Department of Commerce, National Oceanic and Atmospheric Administration. <https://gml.noaa.gov/ccgg/trends/>

Pro Oxygen. (2025). *CO₂ ice core data*. CO₂.Earth. <https://www.CO2.earth/CO2-ice-core-data>

U.S. Energy Information Administration. (n.d.). *Annual energy review: U.S. energy consumption by source*. <https://www.eia.gov/>

[Webb, E. \(2024, March 4\). *Ensemble Oceanic Niño Index*. Webber Weather.
 https://www.webberweather.com/ensemble-oceanic-Niño-index.html](https://www.webberweather.com/ensemble-oceanic-Niño-index.html)

Declaration of AI Use

During the preparation of this paper, the author used Google Gemini 3.0 and OpenAI's ChatGPT for assistance with language refinement, grammar correction, and overall clarity of the writing. These tools were also employed to verify the interpretation of model reports. The author reviewed, edited, and validated all content (over and over again) generated or suggested by the AI tools and takes full responsibility for the accuracy and integrity of the final publication.

Group 3 Appendix

Kamran Shirazi & Taylor Kirk

2025-12-08

Table of contents

Introduction	1
Project Problem/Statement	2
Data Overview	2
Convert to Tsibble and Temperatures	3
Exploratory Data Analysis of Global Anomalies	4
Exploratory Data Analysis of Global Temperatures	8
Modeling / Forecasting	11
Monthly_Anomaly Modeling	11
Temperature Modeling	13
ETS	14
TSLM	18
ARIMA	21
Compare Best Models	26
TSLM External Predictors	29
ARIMA External Predictors	40
External Predictor Feature Engineering Steps	55

Introduction

The Industrial Revolution has been a boon to humanity, leading to longer life spans, increased wealth, easy access to clean water, it gave us Netflix...but with every benefit there comes a cost, and the main cost with expansion of human flourishing has been what we now call

climate change. This term encapsulates a host of changes to our environment such as rising sea levels, more extreme weather events, reduction in biodiversity, etc. The climate changing isn't particularly new, the climate is always changing. What's unique about this period in Earth's history is the rate at which everything is changing. And the consensus is that rate of change is driven by human activity dumping CO2 emissions into the atmosphere, driving up CO2 concentration in the atmosphere, causing the global temperature to rise and leading to these rapid ecosystem alterations.

Project Problem/Statement

This project was inspired by a desire to get involved in understanding the scope of the problem and hopefully finding a way to solve or mitigate it. Specifically we wanted to find a way to accurately isolate the impact of CO2 emissions on the increasing temperature trend, and by extension an individuals (defined as nation, state, city, organization etc) impact on the warming trend. The belief is that by quantifying impact, people will be better equipped to take action on changing their behavior and upgrading systems in ways that slow or reverse the warming trends.

Data Overview

```
df <- read_csv("data/global_temp.csv")  
  
glimpse(df, width = 45)
```

```
Rows: 2,109  
Columns: 12  
$ Year          <dbl> 1850, 1850, 1850~  
$ Month         <dbl> 1, 2, 3, 4, 5, 6~  
$ Monthly_Anomaly <dbl> -0.469, -0.660, ~  
$ Monthly_Unc    <dbl> 0.662, 0.456, 0.~  
$ Annual_Anomaly  <dbl> NA, NA, NA, NA, ~  
$ Annual_Unc      <dbl> NA, NA, NA, NA, ~  
$ FiveYear_Anomaly <dbl> NA, NA, NA, NA, ~  
$ FiveYear_Unc    <dbl> NA, NA, NA, NA, ~  
$ TenYear_Anomaly <dbl> NA, NA, NA, NA, ~  
$ TenYear_Unc     <dbl> NA, NA, NA, NA, ~  
$ TwentyYear_Anomaly <dbl> NA, NA, NA, NA, ~  
$ TwentyYear_Unc  <dbl> NA, NA, NA, NA, ~
```

```
summary(df[3:6], digits = 2)
```

Monthly_Anomaly	Monthly_Unc	Annual_Anomaly	Annual_Unc
Min. :-0.8130	Min. :0.020	Min. :-0.5640	Min. :0.015
1st Qu.: -0.2850	1st Qu.:0.069	1st Qu.: -0.2780	1st Qu.:0.037
Median :-0.0870	Median :0.156	Median :-0.1060	Median :0.096
Mean : 0.0069	Mean :0.182	Mean : 0.0052	Mean :0.110
3rd Qu.: 0.1990	3rd Qu.:0.262	3rd Qu.: 0.1797	3rd Qu.:0.168
Max. : 1.4490	Max. :0.770	Max. : 1.3050	Max. :0.360
		NA's :11	NA's :11

The data from Berkely Earth comes in the form of monthly anomalies. They use a baseline period (1950-1980) and record temperature observations as differences from the average temp (by month) of this period. They include the uncertainties, and 5, 10, and 20 year lag periods. For our purposes, we're only concerned with the `Monthly_Anomaly` column

Convert to Tibble and Temperatures

```
Anomaly_data <- df |> mutate(
  Month = month.abb[Month],
  Date = yearmonth(paste(Month, Year)),
  Monthly_Anomaly = replace_na(Monthly_Anomaly, mean(Monthly_Anomaly, na.rm =
    T))) |>
select(c(Date, Monthly_Anomaly)) |>
as_tsibble(index = Date)

baseline_vector <- c(
  '1' = 12.30, '2' = 12.50, '3' = 13.13, '4' = 14.06,
  '5' = 15.00, '6' = 15.66, '7' = 15.90, '8' = 15.75,
  '9' = 15.18, '10' = 14.27, '11' = 13.28, '12' = 12.57
)

converted_df <- Anomaly_data |>
mutate(
  month_char = as.character(month(Date)),
  baseline_temp = baseline_vector[month_char],
  actual_temp = baseline_temp + Monthly_Anomaly
) |>
select(c(Date, actual_temp))
```

```
converted_df |>
  head() |>
  kable()
```

Date	actual_temp
1850 Jan	11.831
1850 Feb	11.840
1850 Mar	12.703
1850 Apr	13.250
1850 May	14.625
1850 Jun	15.298

Reporting in monthly anomalies is common for publications given that the main interest is usually in the overall trend of warming. However for our purposes, since we are attempting to isolate the human effect on the warming trend, we also need to be able to accurately model temperature separate of human impact. The seasonality of global temperatures is a big part of that and using only the monthly Monthly_Anomaly measurements largely strips out that component. Both sets of data will be explored and modeled however to see which one is most useful and it may turn out that each is valuable in different capacities.

Exploratory Data Analysis of Global Anomalies

```
glimpse(df, width = 50)
```

```
Rows: 2,109
Columns: 12
$ Year      <dbl> 1850, 1850, 1850, 185~
$ Month     <dbl> 1, 2, 3, 4, 5, 6, 7, ~
$ Monthly_Anomaly <dbl> -0.469, -0.660, -0.42~
$ Monthly_Unc <dbl> 0.662, 0.456, 0.611, ~
$ Annual_Anomaly <dbl> NA, NA, NA, NA, NA, --~
$ Annual_Unc <dbl> NA, NA, NA, NA, NA, 0~
$ FiveYear_Anomaly <dbl> NA, NA, NA, NA, NA, N~
$ FiveYear_Unc <dbl> NA, NA, NA, NA, NA, N~
$ TenYear_Anomaly <dbl> NA, NA, NA, NA, NA, N~
$ TenYear_Unc <dbl> NA, NA, NA, NA, NA, N~
```

```
$ TwentyYear_Anomaly <dbl> NA, NA, NA, NA, NA, N~
$ TwentyYear_Unc      <dbl> NA, NA, NA, NA, NA, N~
```

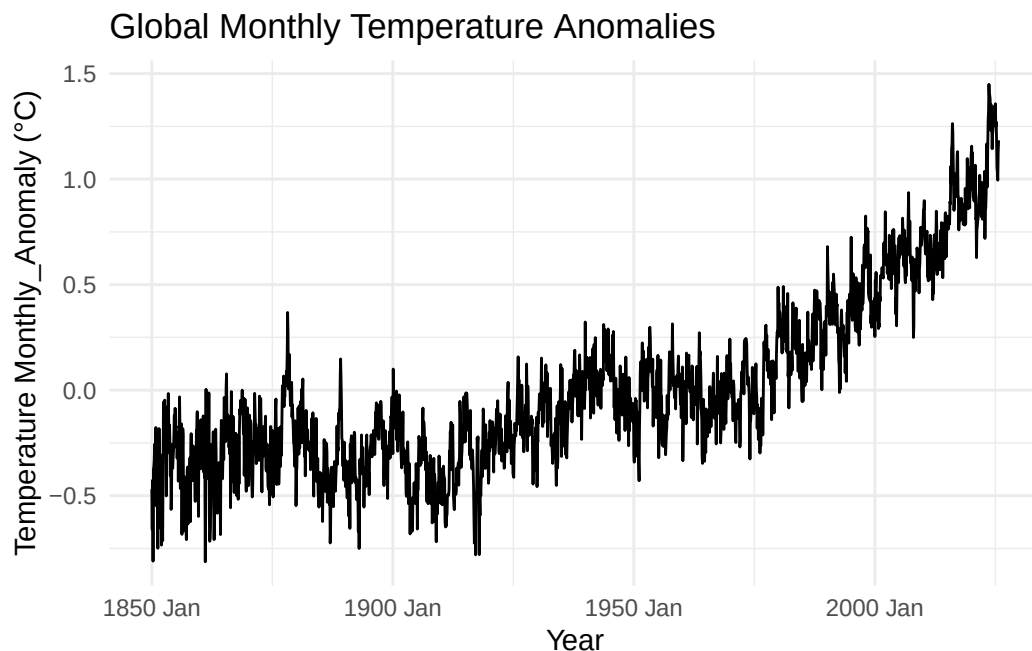
```
names(df)
```

```
[1] "Year"           "Month"          "Monthly_Anomaly"
[4] "Monthly_Unc"    "Annual_Anomaly" "Annual_Unc"
[7] "FiveYear_Anomaly" "FiveYear_Unc"   "TenYear_Anomaly"
[10] "TenYear_Unc"    "TwentyYear_Anomaly" "TwentyYear_Unc"
```

This dataset contains 2,109 monthly observations from 1850–2025, with global temperature anomalies and uncertainties, where `Monthly_Anomaly` is the primary usable series and most multi-year `Monthly_Anomaly` fields contain NA values except at the end of their smoothing windows.

```
train <- Anomaly_data|> filter(Date < yearmonth("2020 Jan"))
test  <- Anomaly_data|> filter(Date >= yearmonth("2020 Jan"))

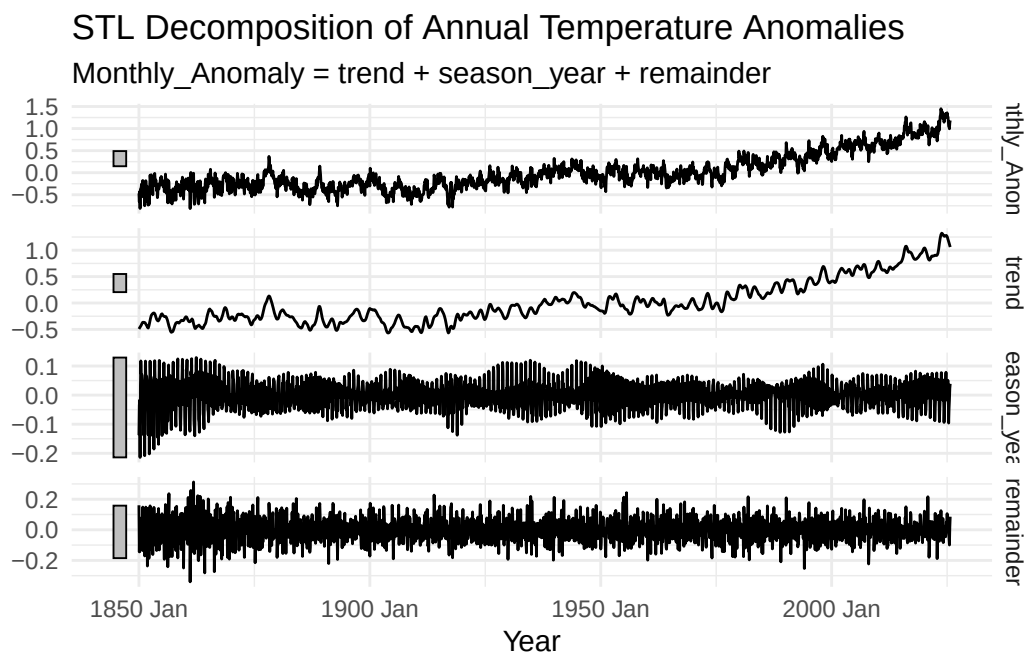
autoplot(Anomaly_data, Monthly_Anomaly) +
  labs(title = "Global Monthly Temperature Anomalies",
       x = "Year", y = "Temperature Monthly_Anomaly (°C)") +
  theme_minimal()
```



The plot shows that what was once mostly cooler-than-average (average as defined as the period between 1950 - 1980) global temperatures has transitioned into a new normal of sustained warming, particularly over the last 40 years. The sharp upward trend in recent decades signals the accelerating pace of climate change.

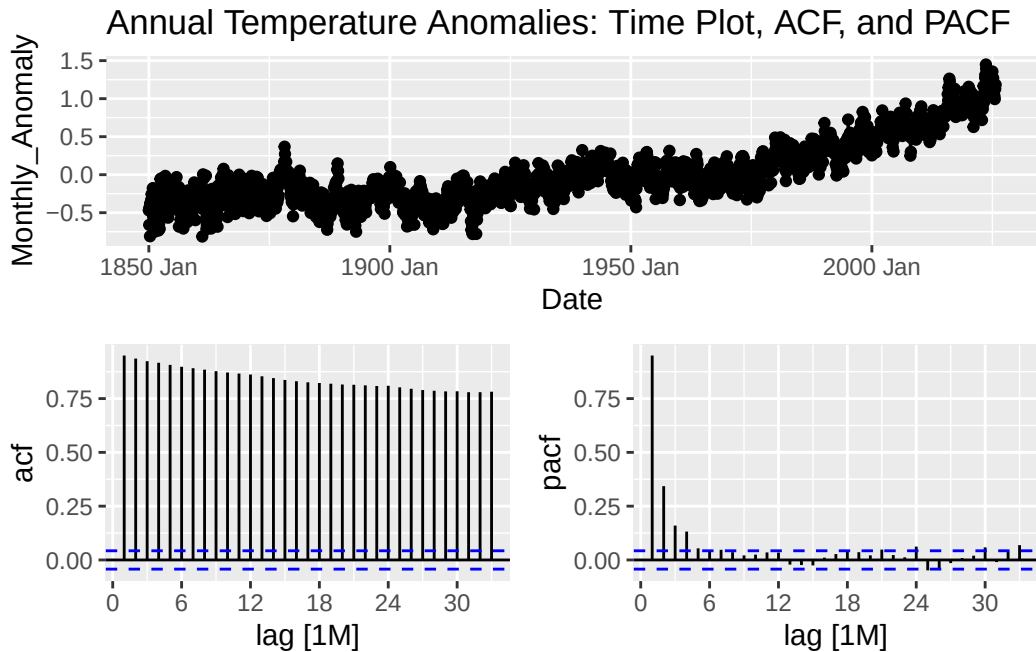
```
climate_stl <- Anomaly_data |>
  model(
    STL(Monthly_Anomaly ~ trend(window = 15))
  ) |>
  components()

autoplot(climate_stl) +
  labs(
    title = "STL Decomposition of Annual Temperature Anomalies",
    x = "Year"
  ) +
  theme_minimal()
```



The STL results highlight how the underlying warming signal has grown stronger over time, while the remainder shows noisy but relatively modest deviations from the trend. This reinforces that annual variability exists but is dwarfed by the long-term increase in global temperatures.

```
Anomaly_data |>
  gg_tsdisplay(Monthly_Anomaly, plot_type = "partial") +
  labs(
    title = "Annual Temperature Anomalies: Time Plot, ACF, and PACF"
  )
```



The climate series exhibits a persistent warming trend, and the ACF's slow decay underscores how each year's temperature is tightly linked to previous years. The PACF's immediate drop-off reinforces that this warming signal creates strong year-to-year inertia in global temperatures.

```
# KPSS
Anomaly_data |>
  features(Monthly_Anomaly, unitroot_kpss) |>
  kable(digits = 4, align = "c")
```

kpss_stat	kpss_pvalue
17.4329	0.01

The KPSS test confirms what the plots already suggest: the `Monthly_Anomaly` series isn't stationary, with the low p -value indicating that the warming trend dominates over random fluctuations.

```
# Non-seasonal differencing needed?
Anomaly_data |>
  features(Monthly_Anomaly, c(unitroot_ndiffs, unitroot_nsdiffs)) |>
  kable(digits = 4, align = "c")
```

ndiffs	nsdifs
1	0

```
Anomaly_data |>
  features(Monthly_Anomaly, guerrero) |>
  kable(digits = 4, align = "c")
```

lambda_guerrero
0.9662

$\lambda \approx 0.966 \rightarrow$ little to no Box-Cox transformation needed; variance is already stable. Unitroot test suggests differencing may be needed

Exploratory Data Analysis of Global Temperatures

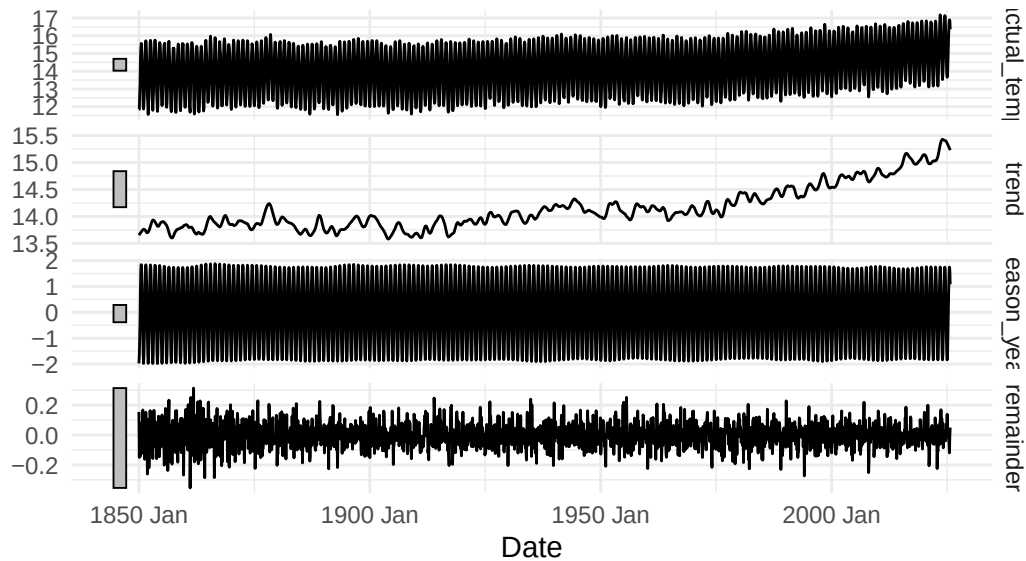
```
summary(converted_df[2])
```

```
actual_temp
Min.      :11.55
1st Qu.   :12.89
Median    :14.17
Mean      :14.14
3rd Qu.   :15.40
Max.      :17.18
```

```
converted_df |>
  model(STL(actual_temp)) |>
  components() |>
  autoplot() +
  labs(title = "STL Decomposition of Global Monthly Actual Temperatures") +
  theme_minimal()
```

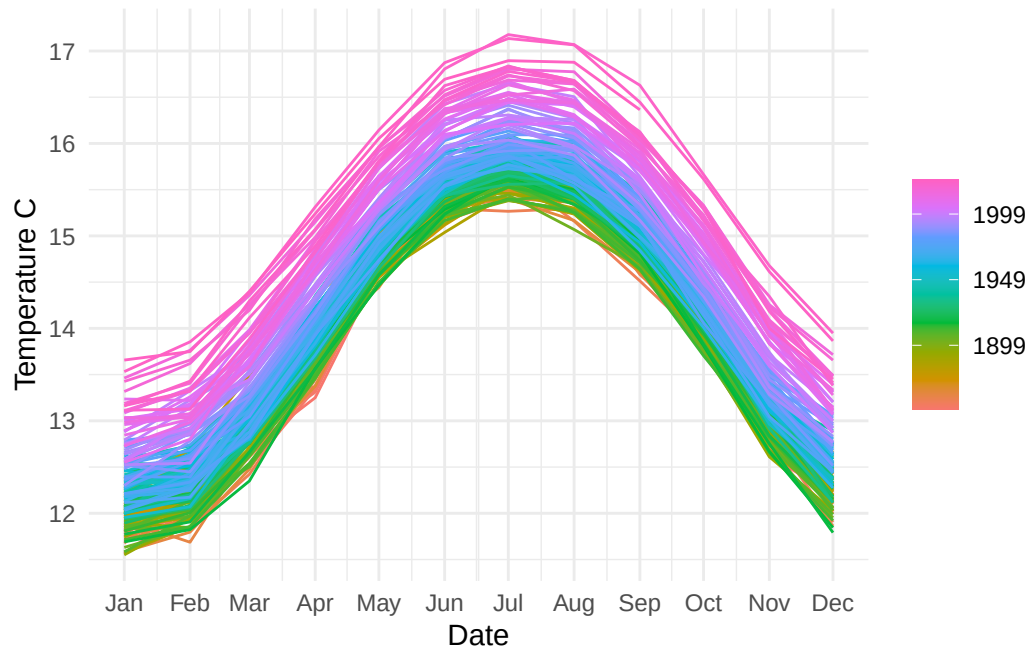
STL Decomposition of Global Monthly Actual Temperatures

$\text{actual_temp} = \text{trend} + \text{season_year} + \text{remainder}$



Decomposition of global temperatures show seasonality to mainly be constant with some compression in later years (could be a result of more precise measurements, could also be a result of warming trends flattening temperature swings each season). The main driver of the changing level is the trend which stayed fairly constant until 1920 when the trend starts to increase, then really takes off after 1980

```
converted_df |>
  ggtime::gg_season() +
  labs(y = "Temperature C") +
  theme_minimal()
```



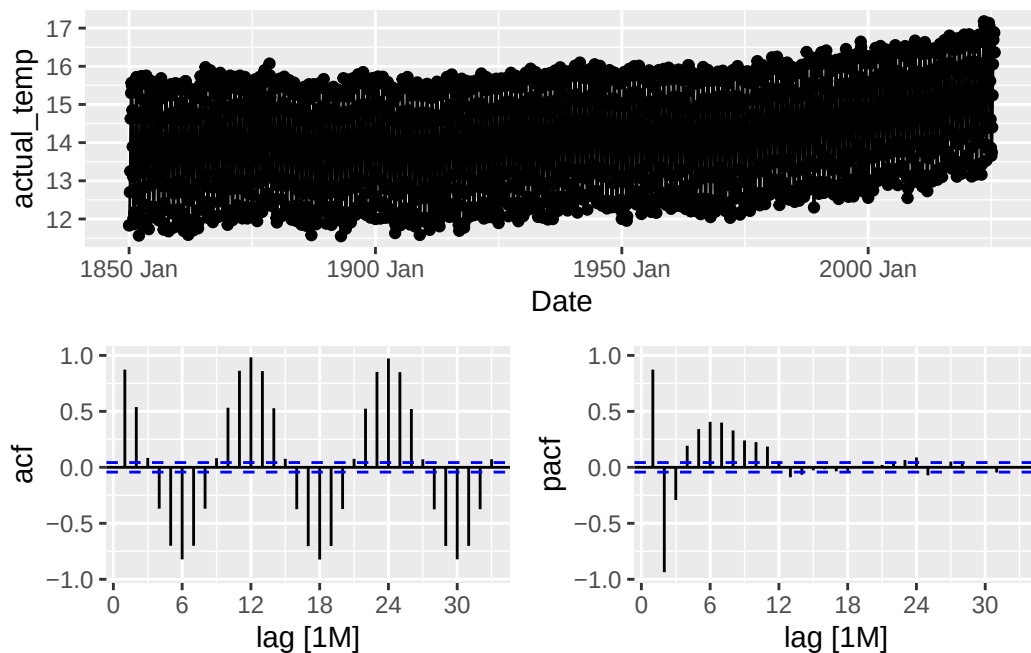
Clearer picture of the increasing trend over time, occurring across all periods

```
converted_df |>
  features(actual_temp, guerrero) |>
  kable(digits = 4, align = "c")
```

lambda_guerrero
1.4379

$\lambda \approx 1.44$, so a power transformation may be helpful

```
gg_tsdisplay(converted_df, actual_temp, plot_type = 'partial')
```



```
converted_df |>
  features(actual_temp, c(unitroot_kpss, unitroot_ndiffs, unitroot_nsdiffs))
  ↪ |>
  kable(digits = 4, align = "c")
```

kpss_stat	kpss_pvalue	ndiffs	nsdifs
8.8813	0.01	1	1

Unitroot tests suggest at least one difference and one seasonal difference. Using `gg_tsdisplay`, the seasonal autocorrelation is obvious, with the summer and winter months grouping together on opposite sides of the line. The pacf shows that the seasonal effects are strong. All of this shows the data is not stationary

Modeling / Forecasting

Monthly_Anomaly Modeling

```
# Fit ETS on the training data
```

```
fit_ets <- train |>  
model(ETS(Monthly_Anomaly))
```

```
# Print model details
```

```
report(fit_ets)
```

Series: Monthly_Anomaly

Model: ETS(A,N,A)

Smoothing parameters:

alpha = 0.4616505

gamma = 0.0001007547

Initial states:

l[0]	s[0]	s[-1]	s[-2]	s[-3]	s[-4]	s[-5]
-0.504211	-0.01987071	-0.01378345	0.03587174	-0.01243867	0.01035226	0.02029624
s[-6]	s[-7]	s[-8]	s[-9]	s[-10]	s[-11]	
0.03041389	0.01181375	-0.004461374	-0.01081024	-0.0249877	-0.02239573	

sigma^2: 0.013

AIC	AICc	BIC
6708.789	6709.026	6793.099

```
fc_ets <- fit_ets |>  
forecast(new_data = test)
```

```
fc_ets_zoom <- fc_ets|>  
filter_index("2020 Jan" ~ "2025 Dec")
```

```
autoplot(  
  fc_ets_zoom,  
  Anomaly_data |> filter_index("2020 Jan" ~ "2025 Dec")  
) +  
labs(  
  title = "Auto-ETS Forecast vs Actuals (2020-2025)",  
  subtitle = "Automatically selected ETS model",  
  x = "Year",  
  y = "Temperature Monthly_Anomaly (°C)"
```

)

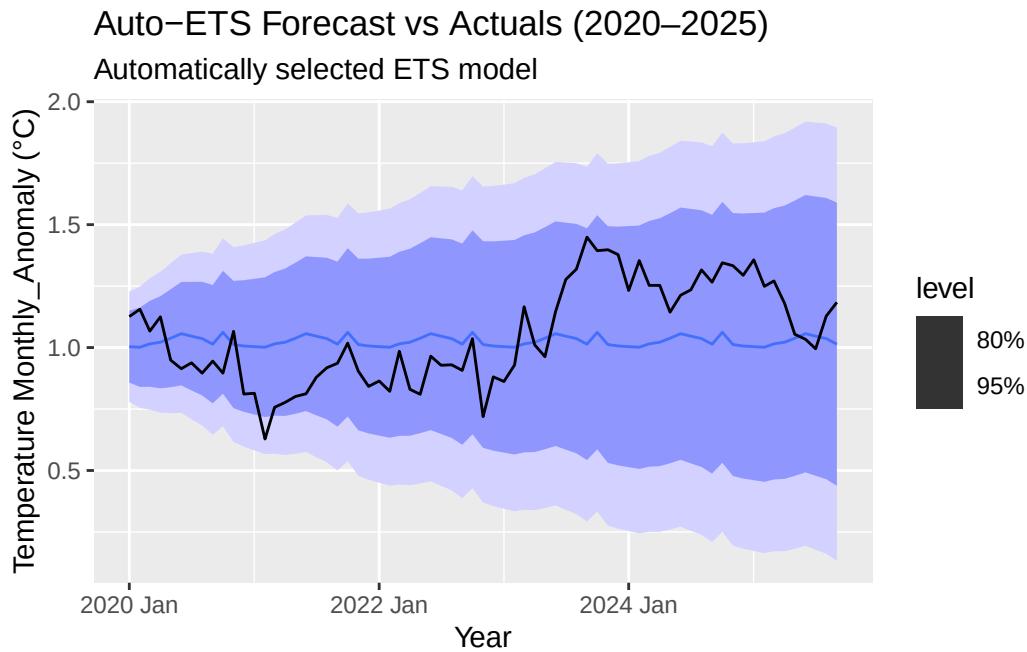


Figure 1: Auto-ETS forecast vs actuals (2020–2025).

The ETS model anticipates a relatively stable `Monthly_Anomaly` level, but the actual values frequently climb toward the upper edge of the 80% and even 95% prediction intervals. This pattern indicates that recent warming spikes are occurring faster and more intensely than the model expects based on historical structure. When observations consistently press against the top of the interval bands, it suggests the model may be underestimating both the trend strength and the volatility of contemporary climate behavior.

Temperature Modeling

```
cv_data <- converted_df |>
  stretch_tsibble(.init = 1506, .step = 60)

cv_trn <- cv_data |>
  group_by(.id) |>
  slice(1:(n() - 60)) |>
```

```

ungroup()

cv_valid <- cv_data |>
  group_by(.id) |>
  slice_tail(n = 60) |>
  ungroup()

```

Modeling on all temp data with cross validation sets created from the first roughly 125 years of data (1850 - 1930) and rolling forward by 5 years (60 months). This creates 11 cv splits covering the range of available observations

ETS

```

ets_fit <- cv_trn |>
  model(
    ets_auto = ETS(),
    additive = ETS(actual_temp ~ error("A") + trend("A") + season("A")),
    multiplicative = ETS(actual_temp ~ error("M") + trend("A") +
      ↪ season("M")),
    damped = ETS(actual_temp ~ error("A") + trend("Ad") + season("A")),
    mult_damp = ETS(actual_temp ~ error("M") + trend("Ad") + season("M")),
    log_auto = ETS(log(actual_temp)),
    box_cox_auto = ETS(box_cox(actual_temp, lambda = 1.5))
  )

```

```

ets_fit |>
  accuracy() |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4, align = "c")

```

.model	RMSE	MAE	ME
box_cox_auto	0.1146	0.0893	5e-04
ets_auto	0.1158	0.0907	8e-04
damped	0.1158	0.0909	7e-04
additive	0.1166	0.0917	-2e-04
mult_damp	0.1195	0.0942	-1e-04

.model	RMSE	MAE	ME
log_auto	0.1198	0.0947	1e-04
multiplicative	0.1249	0.0988	-4e-04

```
# Smoothing parameters
ets_fit |>
  tidy() |>
  group_by(.model, term) |>
  summarise(across(.cols = estimate, .fns = mean)) |>
  pivot_wider(names_from = term, values_from = estimate) |>
  select(.model, alpha, beta, gamma, phi) |>
  kable(digits = 4, align = "c")
```

.model	alpha	beta	gamma	phi
additive	0.4026	0.0001	0.0285	NA
box_cox_auto	0.4386	0.0006	0.0001	0.9727
damped	0.4325	0.0002	0.0178	0.9787
ets_auto	0.4304	0.0003	0.0109	0.9800
log_auto	0.4139	0.0001	0.0343	0.9754
mult_damp	0.4469	0.0001	0.0295	0.9786
multiplicative	0.2812	0.0057	0.0755	NA

```
# Report on top performing model
ets_fit |>
  filter(.id == 10) |>
  select(log_auto) |>
  report()
```

Series: actual_temp

Model: ETS(A,A,A)

Transformation: log(actual_temp)

Smoothing parameters:

alpha = 0.408761

beta = 0.0001000166

gamma = 0.0549685

Initial states:

l[0]	b[0]	s[0]	s[-1]	s[-2]	s[-3]	s[-4]
2.615744	6.255243e-05	-0.1166191	-0.05617134	0.01551034	0.07698261	0.1148955

```

      s[-5]      s[-6]      s[-7]      s[-8]      s[-9]      s[-10]      s[-11]
0.1262374 0.111805 0.06813664 -0.001073793 -0.0717811 -0.1219244 -0.1459979

sigma^2: 1e-04

      AIC      AICc      BIC
-3724.686 -3724.375 -3629.590

```

All versions of ETS performed well on the training data with `box_cox_auto` performing the best. Auto selection chose an additive model with a damped trend. Looking the parameters, the **alpha** parameter values ranged from .2812 to .4469. This shows that the level is fairly stable and perhaps indicates that the recent warming trend is being masked by the long tail of relatively stable temperatures. This idea is bolstered by the the really small, almost 0 values of the **beta** parameter which indicates a stable trend. The **gamma** parameters are all very small as showing, suggesting that the ETS models are correctly reading that the seasonal pattern is relatively stable.

```

ets_fc <- ets_fit |>
  forecast(new_data = cv_valid)

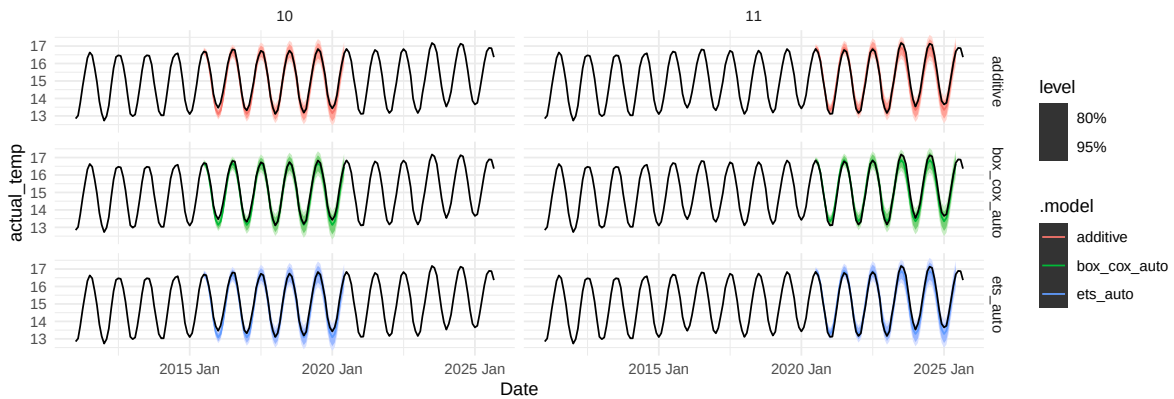
ets_fc |>
  accuracy(cv_valid) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

```

.model	RMSE	MAE	ME
additive	0.1724	0.1412	-0.0013
ets_auto	0.1726	0.1409	0.0239
box_cox_auto	0.1730	0.1409	0.0183
damped	0.1735	0.1419	0.0208
log_auto	0.1758	0.1441	0.0033
mult_damp	0.1763	0.1442	0.0261
multiplicative	0.1919	0.1627	-0.0179

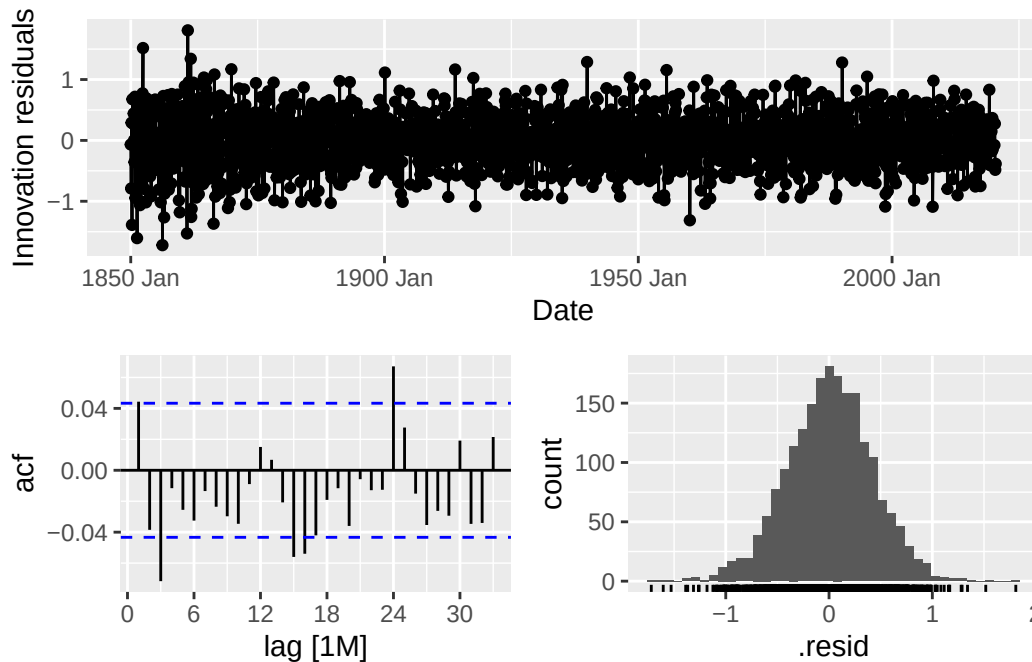
The additive model performs the best on the validation data but the out performance is negligible to `ets_auto` and `box_cox_auto`

```
ets_fc |>
  filter(.id %in% c(10, 11), .model %in% c('additive', 'box_cox_auto',
    ↪ 'ets_auto')) |>
  autoplot() +
  autolayer(converted_df |> filter(year(Date) > 2010), actual_temp) +
  facet_grid(.model ~ .id) +
  theme_minimal()
```



Visualizing the top 3 models illustrates how ETS models do just fine at predicting the fall and spring months, but consistently undershoot the seasonal peaks and exhibit wide prediction intervals around those areas. ETS models have trouble capturing the increasing trend

```
ets_fit |>
  select(box_cox_auto) |>
  tail(n = 1) |>
  ggtime::gg_tsresiduals()
```



Using box cox and checking the residuals of the model fitted on the most data, the residuals show a mostly normal distribution. Slight skew to the right. There are autocorrelations at lag 3 and lag 24. We can confirm that the residuals aren't white noise with a Ljung-Box test. There are certainly factors that influence temperature that ETS isn't capturing

```
ets_fit |>
  augment() |>
  filter(.model == "box_cox_auto") |>
  features(.innov, ljung_box, lag = 24) |>
  summarize(across(.cols = lb_pvalue, .fns = mean)) |>
  kable(digits = 4, align = "c")
```

lb_pvalue
0.0041

TSLM

```
tslm_fit <- cv_trn |>
  model(
```

```

tslm_auto = TSLM(actual_temp),
tslm_trend = TSLM(actual_temp ~ trend()),
tslm_trend_season = TSLM(actual_temp ~ trend() + season()),
tslm_fourier = TSLM(actual_temp ~ trend() + fourier(K = 2)),
tslm_log = TSLM(log(actual_temp) ~ trend() + season()),
tslm_box_cox = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend() +
  ↪ season()),
tslm_piecewise = TSLM(actual_temp ~ trend(knots = c(1920, 1975)) +
  ↪ season())
)

```

Testing Fourier terms for a simpler model as opposed to using season dummy variables, and piece-wise to capture changes in the trend at those points in the data.

```

tslm_fit |>
  accuracy() |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

```

.model	RMSE	MAE	ME
tslm_piecewise	0.1861	0.1492	0.0000
tslm_trend_season	0.1861	0.1492	0.0000
tslm_box_cox	0.1861	0.1493	-0.0006
tslm_log	0.1874	0.1501	0.0013
tslm_fourier	0.1880	0.1508	0.0000
tslm_trend	1.3262	1.1887	0.0000
tslm_auto	1.3409	1.1979	0.0000

```

tslm_fit |>
  select(tslm_trend_season) |>
  tail(n = 1) |>
  report()

```

Series: actual_temp

Model: TSLM

Residuals:

Min	1Q	Median	3Q	Max
-0.63061	-0.15153	-0.01237	0.14543	0.79855

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.172e+01	1.870e-02	626.999	<2e-16 ***
trend()	5.171e-04	8.187e-06	63.157	<2e-16 ***
season()year2	2.096e-01	2.365e-02	8.861	<2e-16 ***
season()year3	8.370e-01	2.365e-02	35.387	<2e-16 ***
season()year4	1.779e+00	2.365e-02	75.227	<2e-16 ***
season()year5	2.735e+00	2.365e-02	115.630	<2e-16 ***
season()year6	3.411e+00	2.365e-02	144.184	<2e-16 ***
season()year7	3.636e+00	2.369e-02	153.494	<2e-16 ***
season()year8	3.479e+00	2.369e-02	146.845	<2e-16 ***
season()year9	2.890e+00	2.369e-02	122.000	<2e-16 ***
season()year10	2.024e+00	2.369e-02	85.447	<2e-16 ***
season()year11	9.840e-01	2.369e-02	41.538	<2e-16 ***
season()year12	2.759e-01	2.369e-02	11.645	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2187 on 2033 degrees of freedom

Multiple R-squared: 0.9743, Adjusted R-squared: 0.9742

F-statistic: 6435 on 12 and 2033 DF, p-value: < 2.22e-16

The `trend_season`, `box_cox` and `piecewise` models all perform the same on the training data. Looking at the model report for the `trend_season` model we can see that all parameters are significant.

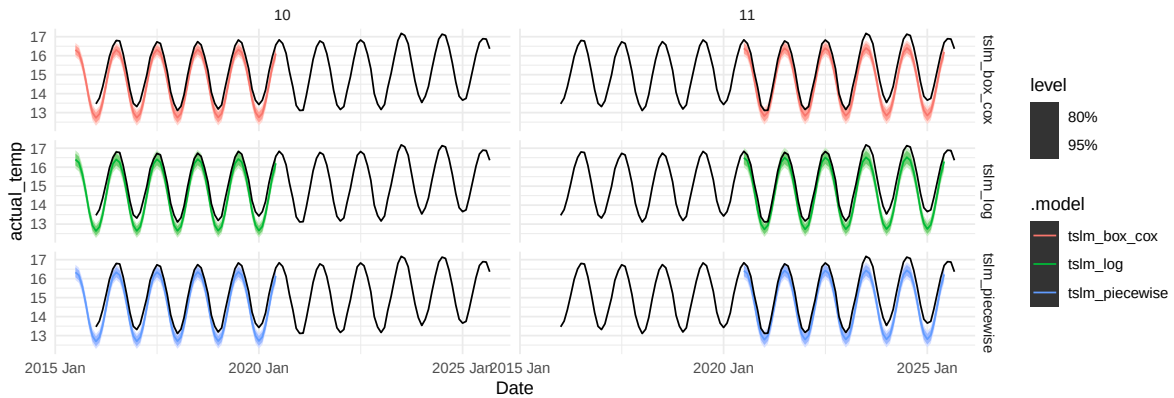
```
tslm_fc <- tslm_fit |>
  forecast(new_data = cv_valid)

tslm_fc |>
  accuracy(cv_valid) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(MAE) |>
  kable(digits = 4)
```

.model	RMSE	MAE	ME
tslm_log	0.3198	0.2881	0.2660
tslm_piecewise	0.3212	0.2933	0.2724
tslm_trend_season	0.3212	0.2933	0.2724
tslm_fourier	0.3222	0.2939	0.2723
tslm_box_cox	0.3212	0.2940	0.2731
tslm_trend	1.3361	1.1945	0.2735
tslm_auto	1.4553	1.2584	0.6108

TSLM log performs the best on the validation data but significantly under-perform ETS models

```
tslm_fc |>
  filter(.id %in% c(10, 11), .model %in% c('tslm_log', 'tslm_piecewise',
    ↪ 'tslm_box_cox')) |>
  autoplot() +
  autolayer(converted_df |> filter(year(Date) > 2015), actual_temp) +
  facet_grid(.model ~ .id) +
  theme_minimal()
```



It's pretty clear from the visuals that TSLM models are failing even harder than the ETS models at capturing the warming trend.

ARIMA

```
trn_data <- converted_df |>
  slice(1:(n() - 60))

valid_data <- converted_df |>
  slice_tail(n = 60)
```

ARIMA models take too long to converge on CV splits and often time out, so testing them first on normal splits

```
arima_fit <- trn_data |>
  model(
    arima_auto = ARIMA(actual_temp),
    arima_box = ARIMA(box_cox(actual_temp, lambda = 1.5)),
    arima_log = ARIMA(log(actual_temp))
  )

arima_fit |>
  accuracy() |>
  select(.model, RMSE, MAE, ME) |>
  arrange(RMSE) |>
  kable(digits = 4)
```

.model	RMSE	MAE	ME
arima_box	0.1191	0.0931	-4e-04
arima_auto	0.1192	0.0932	0e+00
arima_log	0.1318	0.1027	2e-04

ARIMA box_cox and auto performed the same on the training data

```
arima_fit |>
  select(arima_box) |>
  report()
```

```
Series: actual_temp
Model: ARIMA(1,0,0)(0,1,1)[12] w/ drift
Transformation: box_cox(actual_temp, lambda = 1.5)
```

```
Coefficients:
      ar1      sma1  constant
```

```

      0.7088 -0.8825  0.0079
s.e.  0.0176  0.0126  0.0012

```

```

sigma^2 estimated as 0.1955:  log likelihood=-1235.64
AIC=2479.28   AICc=2479.3   BIC=2501.76

```

```

arima_fit |>
  select(arima_auto) |>
  report()

```

```

Series: actual_temp
Model: ARIMA(1,0,0)(0,1,1)[12] w/ drift

```

```

Coefficients:
      ar1      sma1  constant
      0.7053 -0.8840   0.0021
s.e.  0.0176  0.0126   0.0003

```

```

sigma^2 estimated as 0.01431:  log likelihood=1426.97
AIC=-2845.94   AICc=-2845.92   BIC=-2823.47

```

Surprisingly the ARIMA models only selected a seasonal differencing instead of both a seasonal and non-seasonal. 1 non-seasonal AR component was selected and 1 seasonal moving average. So the auto selected models are using the previous observation and the previous seasons forecast error to make its predictions.

```

arima_fc <- arima_fit |>
  forecast(new_data = valid_data)

arima_fc |>
  accuracy(valid_data) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

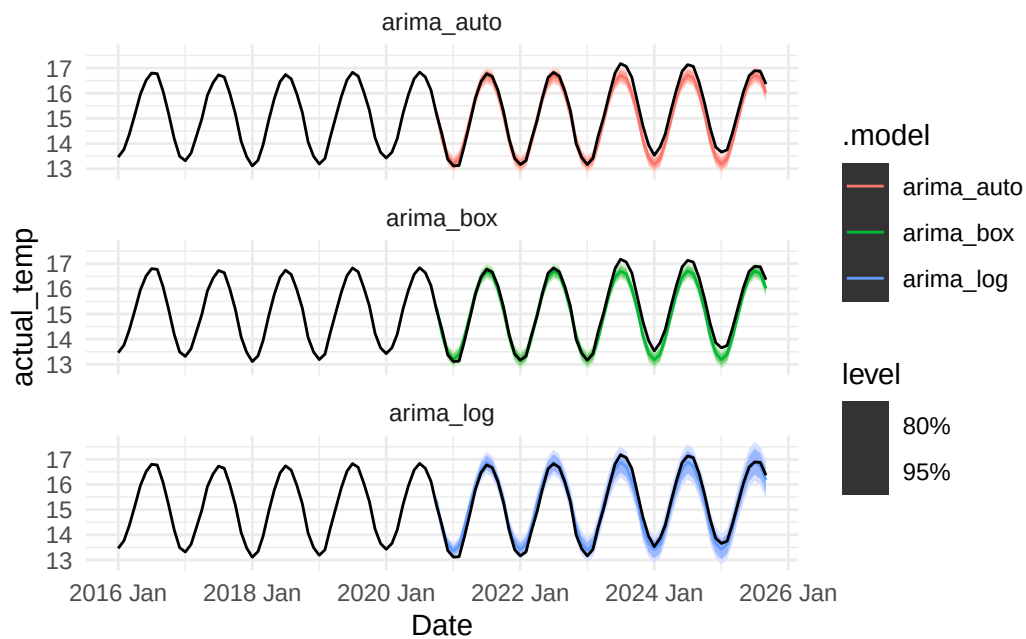
```

.model	RMSE	MAE	ME
arima_log	0.2174	0.1823	0.0214
arima_box	0.2929	0.2365	0.2020

.model	RMSE	MAE	ME
arima_auto	0.2929	0.2357	0.2035

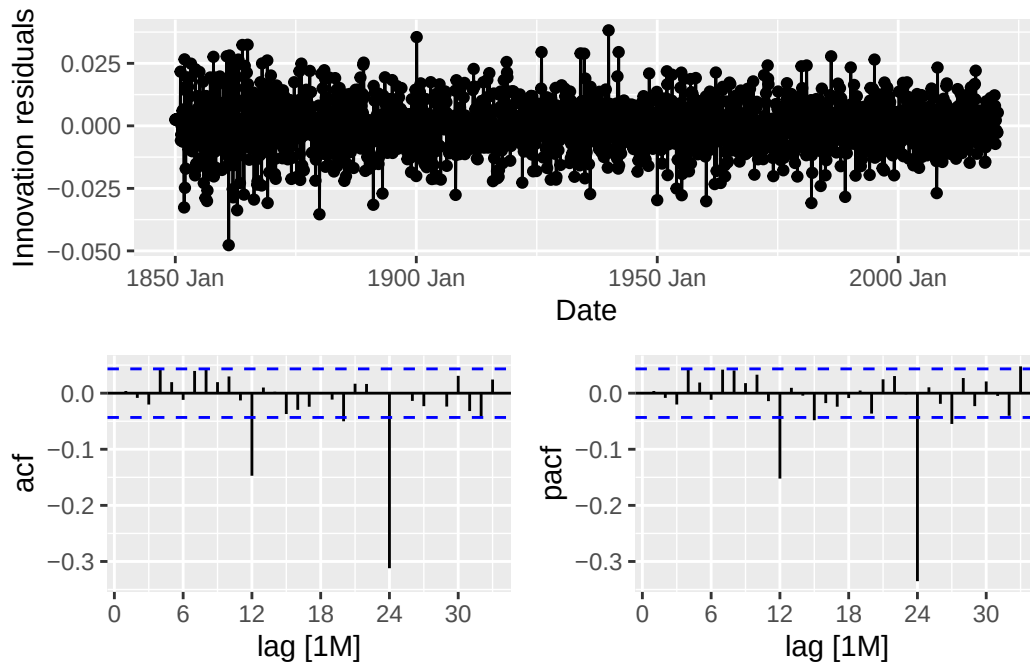
Arima log does best on the validation data by a large margin.

```
arima_fc |>
  autoplot() +
  autolayer(converted_df |> filter(year(Date) > 2015), actual_temp) +
  facet_wrap(~ .model, ncol = 1) +
  theme_minimal()
```



We see that the ARIMA log model does slightly better at capturing the peaks than the other two, but all still don't quite capture the increasing trend.

```
arima_fit |>
  select(arima_log) |>
  ggtime::gg_tsresiduals(plot_type = 'partial')
```



```

arima_fit |>
  augment() |>
  filter(.model == "arima_box") |>
  features(.innov, ljung_box, lag = 24) |>
  kable(digits = 4, align = "c")

```

.model	lb_stat	lb_pvalue
arima_box	229.746	0

Residuals aren't even close to stationary. With ARIMA we can experiment with other parameter values

```

arima_fit <- trn_data |>
  model(
    arima_diff = ARIMA(actual_temp ~ pdq(d = 1)),
    arima_box_diff = ARIMA(box_cox(actual_temp, lambda = 1.5) ~ pdq(d = 1)),
    arima_box_ar_two = ARIMA(box_cox(actual_temp, lambda = 1.5) ~ pdq(p =
      ↪ 2)),
    arima_custom = ARIMA(actual_temp ~ 0 + pdq(3, 1, 2) + PDQ(1, 1, 2))
  )

```

```

arima_fc <- arima_fit |>
  forecast(new_data = valid_data)

arima_fc |>
  accuracy(valid_data) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

```

.model	RMSE	MAE	ME
arima_custom	0.2110	0.1714	0.0801
arima_diff	0.2169	0.1814	-0.1230
arima_box_diff	0.2183	0.1818	-0.1199
arima_box_ar_two	0.2212	0.1851	0.0201

Adding non-seasonal differencing, another AR term and a seasonal MA term to deal with the autocorrelation at lag 24 helped improved model results on the validation data

```

arima_fit |>
  augment() |>
  filter(.model == "arima_custom") |>
  features(.innov, ljung_box, lag = 24) |>
  kable(digits = 4, align = "c")

```

.model	lb_stat	lb_pvalue
arima_custom	21.0759	0.6342

A high p-value tells us that the residuals are now resembling white noise

Compare Best Models

```

compare_fit <- cv_trn |>
  model(
    ETS = ETS(actual_temp ~ error("A") + trend("A") + season("A")),

```

```

    TSLM = TSLM(log(actual_temp) ~ trend() + season()),
    ARIMA = ARIMA(actual_temp ~ 0 + pdq(3, 1, 2) + PDQ(1, 1, 2))
  )

compare_fc <- compare_fit |>
  forecast(new_data = cv_valid)

compare_fc |>
  accuracy(cv_valid) |>
  filter(.id != 6) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

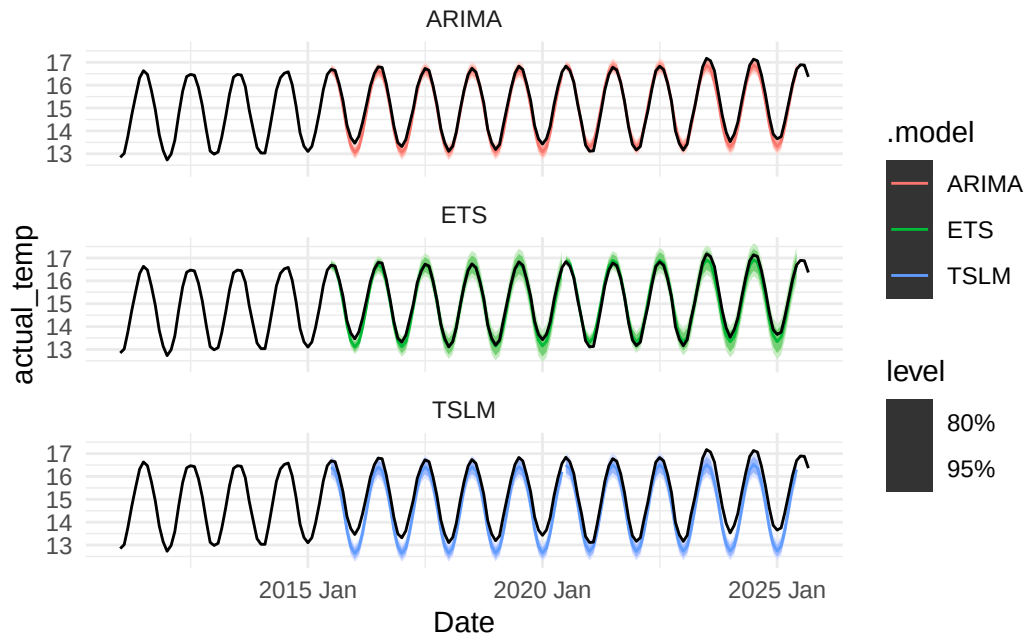
```

.model	RMSE	MAE	ME
ARIMA	0.1634	0.1323	0.0482
ETS	0.1748	0.1429	-0.0004
TSLM	0.3191	0.2880	0.2637

```

compare_fc |>
  filter(.id %in% c(10, 11)) |>
  autoplot() +
  autolayer(converted_df |> filter(year(Date) > 2010), actual_temp) +
  facet_wrap(~ .model, ncol = 1) +
  theme_minimal()

```



The metrics based on RMSE show that ARIMA does slightly better than ETS and both do much better than TSLM. Comparing ARIMA to ETS visually, we see much more uncertainty in the ETS predictions. Both models still seem to under shoot the peaks which is a strong indicator that other factors are at play that will need to be considered. In addition, looking at the `mable` for the arima models shows a null result on split 6 so there may be some instability in the chosen parameters

```
compare_fit |>
  select(ARIMA)
```

```
# A mable: 11 x 2
# Key:      .id [11]
   .id      ARIMA
  <int>    <model>
1     1 <ARIMA(3,1,2)(1,1,2)[12]>
2     2 <ARIMA(3,1,2)(1,1,2)[12]>
3     3 <ARIMA(3,1,2)(1,1,2)[12]>
4     4 <ARIMA(3,1,2)(1,1,2)[12]>
5     5 <ARIMA(3,1,2)(1,1,2)[12]>
6     6 <NULL model>
7     7 <ARIMA(3,1,2)(1,1,2)[12]>
8     8 <ARIMA(3,1,2)(1,1,2)[12]>
9     9 <ARIMA(3,1,2)(1,1,2)[12]>
```

10 10 <ARIMA(3,1,2)(1,1,2)[12]>
11 11 <ARIMA(3,1,2)(1,1,2)[12]>

TSLM External Predictors

According to the Intergovernmental Panel on Climate Change (IPCC, 2021), the main factors that influence global temperature are greenhouse gas concentrations, natural weather and ocean-atmosphere patterns, and variations in solar irradiance. To see which drivers best serve our forecasting purpose, we can model these factors individually and compare their predictive performance. Of the below predictors, accurate monthly estimates are available across varying time periods. The latest are for CO₂ (1958), CH₄ and N₂O (1977). CO₂ is a more significant predictor than the others so the cutoff date for the training data used was 1958. Techniques used to simulate the monthly estimates for N₂O and CH₄ between 1958 and 1977 will be discussed below

<https://www.ipcc.ch/report/ar6/wg1/>

TSI

TSI is a measure of energy that the Earth receives from the Sun across all lengths of electromagnetic waves and plays a small but meaningful role in global temperatures. The measurement is taken at the top of Earth's atmosphere and is reported in Watts per square meter. The data from this predictor comes from <https://www.ncei.noaa.gov/products/climate-data-records/total-solar-irradiance> and is available in monthly observations dating back to 1874.

ENSO Index

ENSO is an acronym for El Niño - Southern Oscillation. The index is the measurement of the difference in sea surface temperatures (SST) from the long-term average (long-term defined as the period between 1991 and 2000) in the equatorial Pacific Ocean (Bureau of Meteorology, n.d.). Warmer than usual temperatures are referred to as El Niño and cooler than average temperatures are referred to as La Niña events. Part of the challenge of sourcing this data is that there are several different indices reported from several different locations around the world. The mostly commonly used one comes from the Niño 3.4 region. For this project the Ensemble Oceanic Niño Index (ENS ONI) is used and can be read about here <https://www.webberweather.com/ensemble-oceanic-nino-index.html>. The data extends back to 1850. The main limitation is that the modern record estimates for the ENSO index began in roughly 1950, so various statistical techniques paired with historical data and documents have been used to record estimates going back further.

In addition to the index itself, various other feature engineering steps were performed. The ENSO index is a monthly measurement so there was concern about noise unduly influencing the models. Three new variables were created with different smoothing intervals (3-month, 6-month and 12-month) to test if smoothing the variables improved model performance. In addition, it's unreasonable to assume that any differences in the ocean temperature today will

immediately effect the global temperature estimates. Indeed, El Nino and La Nina events are defined as 5 consecutive overlapping 3-month periods of temperatures above or below a 0.5 degree threshold <https://ggweather.com/enso/oni.html>. So another variable was created that took the 3-month smooth ENSO Index and added a 4-month lag.

Lastly, instead of using the numerical index, dummy variables were created to indicate the presence of an El Nino or La Nina weather event for the relevant time period.

CO2

It's well known that Greenhouse Gasses (GHG's) are the main drivers of global warming, with CO2 making up the majority of emissions. The most reliable CO2 concentration observations are reported in parts per million (ppm) and are recorded at the <https://gml.noaa.gov/ccgg/trends/>. The Manua Loa Observatory did not begin recording CO2 concentration data until 1958. However, annual atmospheric CO2 estimates derived from drilled ice cores date back to one million years. To engineer monthly estimates of CO2 concentration, a spline regression was performed on the annual estimates from 1850-1957 to smooth out the step curve. The seasonal variation was then engineered by taking the average seasonal variation of the Mauna Loa observations, scaling by a factor to account for the lower historical concentration, then adding that back to the engineered monthly observation.

As atmospheric concentration of CO2 can take time to show up in global temperatures, in addition to the current observation of CO2 being used, lagged versions were also engineered for periods of 1, 3, 5 and 10 years.

RF_CO2

Radiative forcing is not a measurement but rather a scientific principle. It is used as a way to quantify how the Earth's balance of energy (in this case measured as Watts per meter squared) changes when another variable changes. For this project, the radiative forcing of CO was measured. It was decided to only use CO as this variable is recognized as having the largest forcing effect of the various physical drivers of temperature change. The equation used to calculate the radiative forcing variable is as follows

$$RF = 5.25 \times \log(C / C_0)$$

Where C_0 is defined as the reference CO concentration level prior to the industrial revolution. 278 ppm is the most widely used value.

Volcanic Activity

Volcanic eruptions can have profound effects on the climate that last anywhere from 1 to 3 years. The National Oceanic and Atmospheric Administration (NOAA) keeps a record of all known volcanic eruptions and their various attributes dating back 1000's of years. Volcanic eruptions are happening constantly. There are 613 recorded events dating back to 1850. The strength of an eruption is measured by a volcanic explosivity index (VEI). Similar to the Richter scale, this index ranges from 0 to 8 and is logarithmic. Measuring the effect size of a volcanic eruption on the overall climate is challenging and depends on several factors. For the

purposes of this project it was decided to use only volcanic events with a VEI of 4 or above. Any lower and it's unlikely the explosion would be large enough to have lasting effects. Then it was a question of how to include those events in the data. A thorough scientific exploration of the exact effect on the climate for each event was beyond the scope of this project. However 3 things are known about volcanic activity. 1) Large eruptions have an impact on the overall climate. 2) These eruptions have effects lasting 1 - 3 years. 3) These effects are not constant over that time period. So to include these events into the data, 2 pseudo-dummy variables were created. A value of 1 was used at the start of the event, and then one variable was subjected to a linear decay over a 3yr time period and another was subject to an exponential decay over the 3 year time period, to simulate the effect of ash and debris dissipating over time. In the event of overlapping eruptions, these effects were added together.

CH4 & N2O

A similar process was followed for these predictors as was used for CO2. The data was sourced from the same place, the only difference is that monthly estimates are not available until 1977

```
predictor_modeling <- read_rds("data/lagged_external_predictors.rds")  
  
length(colnames(predictor_modeling))
```

```
[1] 30
```

After feature engineering, the dataset now has 30 columns. The code used to engineer the external predictors can be found [at the end](#)

There are over 100 total combinations of external predictors, so instead of trying all of them, a handful of complementary ones are tested and reviewed to see if any other combinations are worth exploring. For example, current co2_ppm combined with la_nina and el_nino events are compared against the same but with a 10yr lag on the co2_ppm variable. If the lagged version of this model performs better, then lagged trends are worth looking further into

```
cv_data <- predictor_modeling |>  
  filter(year(Date) > 1957) |>  
  stretch_tsibble(.init = 573, .step = 60)  
  
cv_trn <- cv_data |>  
  group_by(.id) |>  
  slice(1:(n() - 60)) |>  
  ungroup()  
  
cv_valid <- cv_data |>
```

```

group_by(.id) |>
slice_tail(n = 60) |>
ungroup()

tslm_regressor_fit <- cv_trn |>
model(
  tslm_trend_season = TSLM(actual_temp ~ trend() + season()),
  tslm_log = TSLM(log(actual_temp) ~ trend() + season()),
  tslm_box_cox = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend() +
    ↪ season()),
  tslm_all = TSLM(actual_temp ~ trend() + season() + el_nino + la_nina +
    ↪ co2_ppm + ch4_ppb + n2o_ppb + TSI + volcano_forcing),
  tslm_all_box = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend() +
    ↪ season() + el_nino + la_nina + co2_ppm + ch4_ppb + n2o_ppb + TSI +
    ↪ volcano_forcing),
  tslm_ch4_co2_nino = TSLM(actual_temp ~ trend() + season() + el_nino +
    ↪ la_nina + co2_ppm + ch4_ppb),
  tslm_co2_nino = TSLM(actual_temp ~ trend() + season() + el_nino + la_nina
    ↪ + co2_ppm),
  tslm_nino = TSLM(actual_temp ~ trend() + season() + el_nino + la_nina),
  tslm_enso = TSLM(actual_temp ~ trend() + season() + ENSO),
  tslm_enso_smooth = TSLM(actual_temp ~ trend() + season() +
    ↪ enso_smooth_12),
  tslm_co2_nino_lag = TSLM(actual_temp ~ trend() + season() + el_nino +
    ↪ la_nina + co2_lag1)
)

tslm_regressor_fit |>
accuracy() |>
group_by(.model) |>
summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
arrange(RMSE) |>
kable(digits = 4)

```

.model	RMSE	MAE	ME
tslm_all_box	0.1188	0.0935	-2e-04
tslm_all	0.1190	0.0936	0e+00
tslm_ch4_co2_nino	0.1214	0.0958	0e+00
tslm_co2_nino	0.1242	0.0981	0e+00
tslm_co2_nino_lag	0.1260	0.0993	0e+00
tslm_enso_smooth	0.1299	0.1023	0e+00

.model	RMSE	MAE	ME
tslm_nino	0.1332	0.1066	0e+00
tslm_enso	0.1368	0.1107	0e+00
tslm_trend_season	0.1461	0.1176	0e+00
tslm_box_cox	0.1465	0.1178	-4e-04
tslm_log	0.1479	0.1191	8e-04

Training metrics show that all the predictors being added leads to the best model, whereas models without any predictors underperform

```
tslm_regressor_fit |>
  glance() |>
  group_by(.model) |>
  summarize(across(.cols = c(AIC, AICc, BIC), .fns = mean)) |>
  arrange(AICc)
```

```
# A tibble: 11 x 4
  .model      AIC  AICc  BIC
  <chr>      <dbl> <dbl> <dbl>
1 tslm_log    -5722. -5721. -5659.
2 tslm_co2_nino -2606. -2605. -2530.
3 tslm_co2_nino_lag -2588. -2587. -2513.
4 tslm_enso_smooth -2555. -2555. -2489.
5 tslm_nino    -2519. -2518. -2448.
6 tslm_enso    -2490. -2489. -2423.
7 tslm_trend_season -2407. -2407. -2345.
8 tslm_all     -2365. -2363. -2274.
9 tslm_ch4_co2_nino -2349. -2348. -2271.
10 tslm_all_box    -868.  -866.  -777.
11 tslm_box_cox    -732.  -731.  -670.
```

Interestingly, tslm_log is the best model by far on the AICc metric, so the gains may not be all that much by adding predictors

```
tslm_regressor_fit |>
  select(tslm_all_box) |>
  tail(n = 1) |>
  report()
```

Series: actual_temp

Model: TSLM

Transformation: box_cox(actual_temp, lambda = 1.5)

Residuals:

	Min	1Q	Median	3Q	Max
	-1.3498	-0.2808	-0.0138	0.3062	1.1927

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-1.829e+02	6.784e+01	-2.696	0.007188	**
trend()	-6.771e-03	1.535e-03	-4.409	1.21e-05	***
season()year2	6.992e-01	8.594e-02	8.136	2.00e-15	***
season()year3	2.899e+00	8.647e-02	33.525	< 2e-16	***
season()year4	6.328e+00	8.815e-02	71.788	< 2e-16	***
season()year5	9.818e+00	8.898e-02	110.341	< 2e-16	***
season()year6	1.257e+01	8.739e-02	143.882	< 2e-16	***
season()year7	1.360e+01	8.564e-02	158.751	< 2e-16	***
season()year8	1.322e+01	8.679e-02	152.368	< 2e-16	***
season()year9	1.096e+01	8.932e-02	122.746	< 2e-16	***
season()year10	7.592e+00	8.963e-02	84.702	< 2e-16	***
season()year11	3.581e+00	8.759e-02	40.887	< 2e-16	***
season()year12	1.017e+00	8.645e-02	11.766	< 2e-16	***
el_nino	1.346e-01	4.439e-02	3.032	0.002526	**
la_nina	-3.062e-01	4.695e-02	-6.523	1.37e-10	***
co2_ppm	7.708e-02	7.967e-03	9.675	< 2e-16	***
ch4_ppb	1.690e-03	5.032e-04	3.358	0.000831	***
n2o_ppb	9.750e-03	1.197e-02	0.815	0.415562	
TSI	1.334e-01	4.994e-02	2.672	0.007721	**
volcano_forcing	-9.692e-02	2.823e-02	-3.434	0.000633	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4599 on 666 degrees of freedom

Multiple R-squared: 0.9921, Adjusted R-squared: 0.9919

F-statistic: 4417 on 19 and 666 DF, p-value: < 2.22e-16

All predictors on the best performing training model show as significant except for N2O

```
tslm_fc <- tslm_regressor_fit |>
  forecast(new_data = cv_valid)
```

```

tslm_fc |>
  accuracy(cv_valid) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

```

.model	RMSE	MAE	ME
tslm_all_box	0.1471	0.1172	-0.0497
tslm_co2_nino_lag	0.1476	0.1166	-0.0234
tslm_all	0.1493	0.1196	-0.0526
tslm_co2_nino	0.1519	0.1203	-0.0409
tslm_enso_smooth	0.1519	0.1254	0.0976
tslm_ch4_co2_nino	0.1531	0.1229	-0.0627
tslm_enso	0.1578	0.1297	0.0987
tslm_nino	0.1705	0.1382	0.1017
tslm_trend_season	0.1746	0.1429	0.0990
tslm_box_cox	0.1761	0.1463	0.1004
tslm_log	0.1766	0.1398	0.0904

TSLM with a box_cox transformation and all of the predictors performed the best on the validation data. The `enso_smooth` variable also performed better than either ENSO or the nino dummy variables. These results indicate that other model combinations are worth exploring.

Adding a year lag to `co2_ppm` provided a bit of a boost on the validation data so some of the lagged variables may have more predictive power and are worth looking into

Interesting to note that adding `ch4_ppb` actually reduced model performance compared to just using `co2_ppm`

```

tslm_second_fit <- cv_trn |>
  model(
    tslm_all_lag_box = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend() +
      ↪ season() + el_nino + la_nina + co2_lag1 + n2o_lag3 + ch4_lag3),
    tslm_nino_five_lag = TSLM(actual_temp ~ trend() + season() + el_nino +
      ↪ la_nina + co2_lag5),
    tslm_enso_co2_lag = TSLM(actual_temp ~ trend() + season() + ENSO +
      ↪ co2_lag1),
    tslm_enso_smooth_lag_all = TSLM(actual_temp ~ trend() + season() +
      ↪ enso_smooth_12 + co2_lag1 + n2o_lag3 + ch4_lag3),
    tslm_all_lag = TSLM(actual_temp ~ trend() + season() + el_nino + la_nina
      ↪ + co2_lag1 + n2o_lag3 + ch4_lag3),

```

```

tslm_nino_10_lag = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend() +
  ↪ season() + el_nino + la_nina + co2_lag10),
tslm_enso_smooth_lag = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend()
  ↪ + season() + enso_smooth_6 + co2_lag10)
)

tslm_second_fc <- tslm_second_fit |>
  forecast(new_data = cv_valid)

tslm_second_fc |>
  accuracy(cv_valid) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

```

.model	RMSE	MAE	ME
tslm_enso_smooth_lag	0.1217	0.0950	-0.0063
tslm_enso_smooth_lag_all	0.1298	0.1028	-0.0476
tslm_enso_co2_lag	0.1374	0.1112	-0.0329
tslm_nino_10_lag	0.1413	0.1101	-0.0029
tslm_nino_five_lag	0.1445	0.1133	0.0057
tslm_all_lag_box	0.1464	0.1144	-0.0394
tslm_all_lag	0.1475	0.1172	-0.0420

The target variable undergoing a box_cox transformation using the smoothed enso index over 6 months and a 10 year lag in CO2 concentration performs the best out of all the models by a fair margin

```

tslm_second_fit |>
  select(tslm_enso_smooth_lag) |>
  tail(n = 1) |>
  report()

```

Series: actual_temp

Model: TSLM

Transformation: box_cox(actual_temp, lambda = 1.5)

Residuals:

Min	1Q	Median	3Q	Max
-1.448628	-0.292156	0.003149	0.296378	1.450508

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.297e+01	1.246e+00	10.410	<2e-16 ***
trend()	2.270e-04	4.573e-04	0.496	0.62
season()year2	7.171e-01	8.137e-02	8.813	<2e-16 ***
season()year3	2.934e+00	8.152e-02	35.994	<2e-16 ***
season()year4	6.402e+00	8.197e-02	78.097	<2e-16 ***
season()year5	9.928e+00	8.222e-02	120.741	<2e-16 ***
season()year6	1.263e+01	8.189e-02	154.285	<2e-16 ***
season()year7	1.362e+01	8.142e-02	167.267	<2e-16 ***
season()year8	1.316e+01	8.157e-02	161.305	<2e-16 ***
season()year9	1.087e+01	8.234e-02	132.039	<2e-16 ***
season()year10	7.474e+00	8.282e-02	90.241	<2e-16 ***
season()year11	3.528e+00	8.216e-02	42.944	<2e-16 ***
season()year12	1.024e+00	8.177e-02	12.526	<2e-16 ***
enso_smooth_6	3.182e-01	2.166e-02	14.694	<2e-16 ***
co2_lag10	4.747e-02	4.146e-03	11.450	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4564 on 738 degrees of freedom

Multiple R-squared: 0.9921, Adjusted R-squared: 0.992

F-statistic: 6628 on 14 and 738 DF, p-value: < 2.22e-16

And we can see from the report that all the predictors used are significant except trend. It could be that smoothing out the enso index and the 10 yr lag of co2_ppm captures most of the trend component

```
tslm_best_fit <- cv_trn |>
  model(
    tslm_enso_6 = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend() +
      ↪ season() + enso_smooth_6 + co2_lag10),
    tslm_enso_12 = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend() +
      ↪ season() + enso_smooth_12 + co2_lag10),
    tslm_no_trend = TSLM(box_cox(actual_temp, lambda = 1.5) ~ season() +
      ↪ enso_smooth_12 + co2_lag10),
    tslm_volcanic = TSLM(box_cox(actual_temp, lambda = 1.5) ~ season() +
      ↪ enso_smooth_12 + co2_lag10 + volcano_forcing)
  )
```

```

tslm_best_fc <- tslm_best_fit |>
  forecast(new_data = cv_valid)

tslm_best_fc |>
  accuracy(cv_valid) |>
  group_by(.model) |>
  summarise(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

```

.model	RMSE	MAE	ME
tslm_no_trend	0.1183	0.0927	-0.0032
tslm_volcanic	0.1191	0.0936	0.0013
tslm_enso_12	0.1211	0.0942	-0.0031
tslm_enso_6	0.1217	0.0950	-0.0063

```

tslm_best_fit |>
  select(tslm_no_trend) |>
  tail(n = 1) |>
  report()

```

Series: actual_temp

Model: TSLM

Transformation: box_cox(actual_temp, lambda = 1.5)

Residuals:

Min	1Q	Median	3Q	Max
-1.628084	-0.287697	0.001792	0.303473	1.553132

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.233e+01	2.424e-01	50.849	<2e-16 ***
season()year2	7.143e-01	8.076e-02	8.844	<2e-16 ***
season()year3	2.923e+00	8.076e-02	36.191	<2e-16 ***
season()year4	6.380e+00	8.078e-02	78.985	<2e-16 ***
season()year5	9.896e+00	8.079e-02	122.495	<2e-16 ***
season()year6	1.260e+01	8.078e-02	155.962	<2e-16 ***
season()year7	1.358e+01	8.076e-02	168.156	<2e-16 ***
season()year8	1.312e+01	8.076e-02	162.517	<2e-16 ***

```

season()year9 1.085e+01 8.077e-02 134.317 <2e-16 ***
season()year10 7.461e+00 8.111e-02 91.985 <2e-16 ***
season()year11 3.522e+00 8.109e-02 43.432 <2e-16 ***
season()year12 1.020e+00 8.108e-02 12.581 <2e-16 ***
enso_smooth_12 3.785e-01 2.500e-02 15.142 <2e-16 ***
co2_lag10      4.966e-02 6.909e-04 71.876 <2e-16 ***

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4532 on 739 degrees of freedom

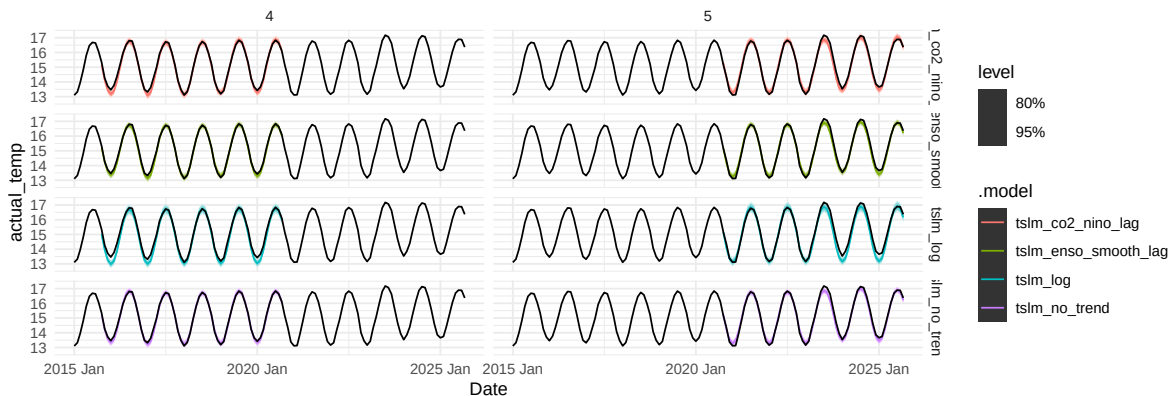
Multiple R-squared: 0.9922, Adjusted R-squared: 0.9921

F-statistic: 7239 on 13 and 739 DF, p-value: < 2.22e-16

```

tslm_fc |>
  bind_rows(tslm_second_fc) |>
  bind_rows(tslm_best_fc) |>
  filter(.id %in% c(4, 5), .model %in% c("tslm_no_trend",
    ↪ "tslm_enso_smooth_lag", "tslm_nino_50_lag", "tslm_co2_nino_lag",
    ↪ "tslm_log")) |>
  autoplot() +
  autolayer(predictor_modeling |> filter(year(Date) >= 2015), actual_temp) +
  facet_grid(.model ~ .id) +
  theme_minimal()

```



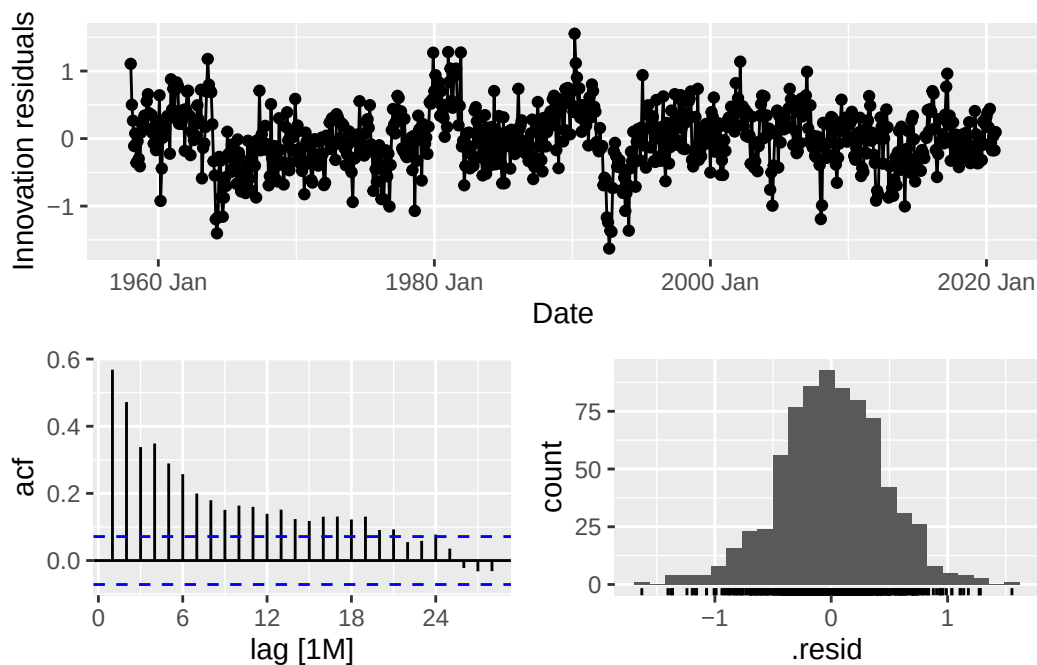
Comparing best models to the under-performing ones, we see that adding predictors, especially ones that are lagged, closes the gap in the increasing trend that other models seem to miss.

Talk more in depth about the AICc scores

```
tslm_best_fit |>
  select(tslm_no_trend) |>
  tail(n = 1) |>
  ggtime::gg_tsresiduals()
```

Registered S3 methods overwritten by 'ggtime':

method	from
+.gg_tsensemble	feasts
autolayer.tbl_ts	fabletools
autoplot.dcmp_ts	fabletools
autoplot.tbl_ts	fabletools
grid.draw.gg_tsensemble	feasts
print.gg_tsensemble	feasts



The biggest issue is that there is significant autocorrelation in the residuals, indicating there is some pattern the TSLM model doesn't catch. We'll see if the ARIMA models are able to accurately account for these factors

ARIMA External Predictors

Using a non-cv split initially to speed up fit time

```

arima_trn <- predictor_modeling |>
  filter(year(Date) > 1957) |>
  slice(1:(n() - 60))

```

```

arima_valid <- predictor_modeling |>
  filter(year(Date) > 1957) |>
  slice_tail(n = 60)

```

```

arima_fit <- arima_trn |>
  model(
    arima_auto = ARIMA(actual_temp ~ co2_ppm + PDQ(D=1)),
    arima_box = ARIMA(box_cox(actual_temp, lambda = 1.5) ~ co2_ppm +
      ↪ PDQ(D=1)),
    arima_log = ARIMA(log(actual_temp) ~ co2_ppm + PDQ(D=1)),
    arima_all = ARIMA(actual_temp ~ co2_ppm + ch4_ppb + n2o_ppb + TSI +
      ↪ el_nino + la_nina + volcano_linear + PDQ(D=1)),
    arima_lag_one = ARIMA(actual_temp ~ co2_lag1 + PDQ(D=1)),
    arima_lag_one_nino = ARIMA(actual_temp ~ co2_lag1 + el_nino + la_nina +
      ↪ PDQ(D=1)),
    arima_enso = ARIMA(actual_temp ~ ENSO + PDQ(D=1)),
    arima_enso_smooth = ARIMA(actual_temp ~ enso_smooth_6 + PDQ(D=1)),
    arima_emissions = ARIMA(actual_temp ~ aggregate_emissions + PDQ(D=1)),
    arima_forcing = ARIMA(log(actual_temp) ~ rf_co2 + PDQ(D=1)),
    arima_enso_delay = ARIMA(actual_temp ~ enso_delay + PDQ(D=1)),
    arima_lag_3 = ARIMA(actual_temp ~ co2_lag3 + PDQ(D=1)),
    arima_physics_robust = ARIMA(log(actual_temp) ~ rf_co2 + volcano_linear +
      ↪ enso_delay + PDQ(D=1)),
    arima_volcano_basic = ARIMA(log(actual_temp) ~ ENSO + co2_ppm +
      ↪ volcano_forcing + PDQ(D = 1))
  )

arima_fit |>
  glance() |>
  arrange(AICc) |>
  select(.model, AICc, AIC, BIC) |>
  kable(align = "c")

```

.model	AICc	AIC	BIC
arima_volcano_basic	-5154.6844	-5154.8811	-5118.0171
arima_physics_robust	-5021.0184	-5021.3198	-4975.2398

.model	AICc	AIC	BIC
arima_forcing	-4925.4545	-4925.6073	-4893.3513
arima_log	-4924.9670	-4925.1637	-4888.2997
arima_enso_smooth	-1259.0458	-1259.2425	-1222.3785
arima_enso	-1254.6394	-1254.8361	-1217.9721
arima_enso_delay	-1227.5044	-1227.7011	-1190.8371
arima_lag_one_nino	-1137.8875	-1138.1888	-1092.1088
arima_auto	-1125.8302	-1126.0269	-1089.1629
arima_emissions	-1125.2980	-1125.4947	-1088.6307
arima_lag_one	-1123.8375	-1124.0343	-1087.1703
arima_lag_3	-1123.6502	-1123.8469	-1086.9829
arima_all	-962.7635	-963.2642	-903.3602
arima_box	827.9954	827.7987	864.6627

Looking through the AICc scores, the log transformations seem to result in the best models

```
arima_fc <- arima_fit |>
  forecast(new_data = arima_valid)

arima_fc |>
  accuracy(arima_valid) |>
  select(.model, RMSE, MAE, ME) |>
  arrange(RMSE) |>
  kable(digits = 4)
```

.model	RMSE	MAE	ME
arima_volcano_basic	0.1444	0.1185	0.0340
arima_physics_robust	0.1551	0.1197	0.0121
arima_all	0.1781	0.1387	-0.0221
arima_lag_one_nino	0.1934	0.1549	0.0475
arima_auto	0.1953	0.1630	0.0072
arima_box	0.1967	0.1638	0.0069
arima_emissions	0.1981	0.1662	0.0081
arima_log	0.1995	0.1654	-0.0205
arima_forcing	0.2022	0.1675	-0.0086
arima_lag_3	0.2067	0.1741	0.0328
arima_lag_one	0.2074	0.1742	0.0301
arima_enso	0.2819	0.2302	0.1923
arima_enso_smooth	0.2859	0.2347	0.2114
arima_enso_delay	0.3086	0.2519	0.2180

.model	RMSE	MAE	ME
--------	------	-----	----

The model including features for volcanic activity and non-lagged variables for CO2 and ENSO is the best overall by any metric. **include a more robust explanation of why a delay in the enso index and why co2 radiative forcing on it's own aren't strong signals**

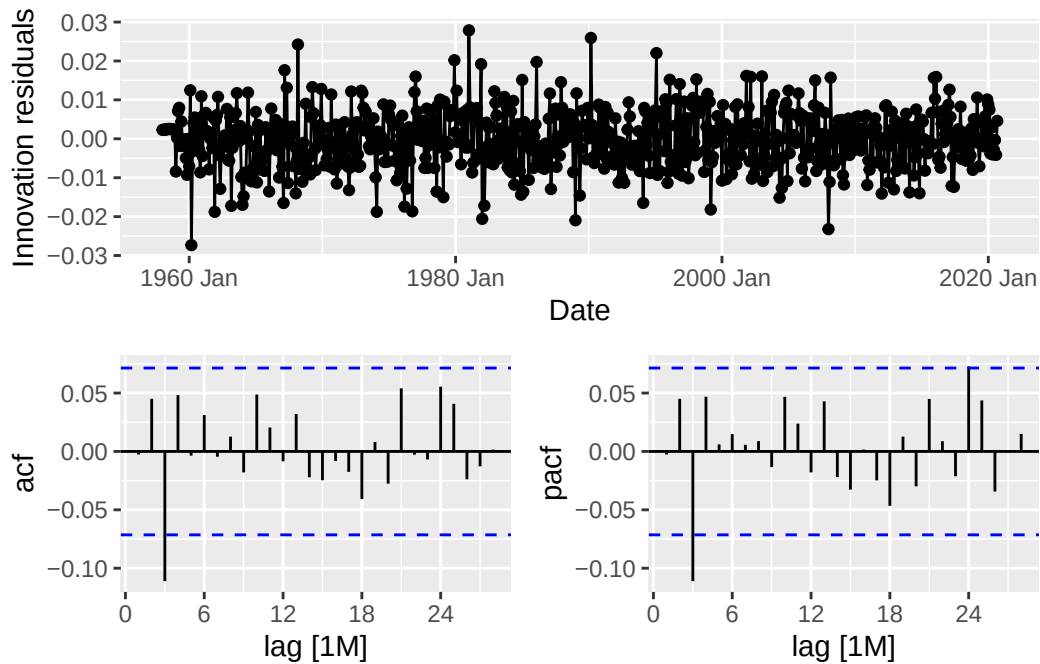
```
# spot check models
arima_fit |>
  select(arima_volcano_basic) |>
  report()
```

```
Series: actual_temp
Model: LM w/ ARIMA(1,0,1)(0,1,2)[12] errors
Transformation: log(actual_temp)
```

```
Coefficients:
      ar1      ma1      sma1      sma2      ENSO      co2_ppm      volcano_forcing
      0.8344 -0.3830 -0.9473  0.0544  0.0046      7e-04      -9e-04
s.e.  0.0326  0.0552  0.0355  0.0364  0.0007      1e-04      9e-04
```

```
sigma^2 estimated as 5.37e-05: log likelihood=2585.44
AIC=-5154.88 AICc=-5154.68 BIC=-5118.02
```

```
arima_fit |>
  select(arima_volcano_basic) |>
  ggtime::gg_tsresiduals(plot_type = "partial")
```



The `arima_volcano_basic` model selected AR(1), MA(1) and sMA(2) with a seasonal difference. The ACF and PACF plots still contain spikes at lag 3. Interestingly, the `arima_physics` robust model contains a spike at lag 24 but not 3. There might be something in delaying the El Nino effect by a few months that explains the autocorrelation

```
# parameter check
arima_fit |>
  tidy() |>
  group_by(term) |>
  summarize(across(.cols = c(estimate:`p.value`), .fns = mean)) |>
  arrange(p.value) |>
  kable(digits = 4)
```

term	estimate	std.error	statistic	p.value
sma1	-0.9267	0.0385	-24.1588	0.0000
sar2	-0.3306	0.0350	-9.4517	0.0000
enso_smooth_6	0.0805	0.0122	6.6143	0.0000
ENSO	0.0313	0.0052	6.3122	0.0000
ar2	0.6779	0.0864	7.8968	0.0000
sar1	-0.6278	0.0347	-18.4039	0.0013
ma2	-0.2650	0.0731	-3.6212	0.0032
enso_delay	0.0158	0.0065	3.7552	0.0120

term	estimate	std.error	statistic	p.value
la_nina	-0.0453	0.0207	-2.1940	0.0297
ma1	0.3425	0.1148	2.8495	0.0433
el_nino	0.0333	0.0177	1.8858	0.0606
aggregate_emissions	0.0006	0.0003	1.8391	0.0663
co2_ppm	0.0126	0.0064	3.2146	0.0665
rf_co2	0.0688	0.0373	1.7833	0.0902
sma2	0.0606	0.0367	1.6484	0.1025
ar3	-0.0228	0.0852	0.1827	0.1074
co2_lag1	0.0079	0.0048	1.6714	0.1114
co2_lag3	0.0069	0.0054	1.2846	0.1993
volcano_linear	-0.0054	0.0062	-1.3169	0.2395
TSI	0.0204	0.0191	1.0671	0.2863
intercept	-0.0011	0.0011	-0.9745	0.3301
volcano_forcing	-0.0009	0.0009	-0.9703	0.3322
ar1	0.1208	0.1062	2.3740	0.4273
n2o_ppb	-0.0049	0.0123	-0.3959	0.6923
ch4_ppb	0.0001	0.0009	0.0822	0.9345

Not necessarily the best way to ascertain how important variables are, but informative nonetheless. ENSO related variables and seasonal ARIMA parameters seem to be the most important to modeling this type of data

Now we'll check the best models from above as well as a few other promising predictor combinations across 5 CV splits and see how they hold up

```

arima_second_fit <- cv_trn |>
  model(
    arima_enso_log_co2 = ARIMA(log(actual_temp) ~ ENSO + co2_ppm + PDQ(D =
      ↪ 1)),
    arima_auto_10 = ARIMA(log(actual_temp) ~ ENSO + co2_lag10 + PDQ(D = 1)),
    arima_physics_robust = ARIMA(log(actual_temp) ~ rf_co2 + volcano_linear +
      ↪ enso_delay + PDQ(D = 1)),
    arima_forcing = ARIMA(log(actual_temp) ~ rf_co2 + enso_smooth_6 + PDQ(D =
      ↪ 1)),
    arima_auto_10_enso_6 = ARIMA(log(actual_temp) ~ enso_smooth_6 + co2_lag10
      ↪ + PDQ(D = 1)),
    arima_robust_enso_6 = ARIMA(log(actual_temp) ~ rf_co2 + volcano_linear +
      ↪ enso_smooth_6 + PDQ(D = 1)),
    arima_co2_10_volcano = ARIMA(log(actual_temp) ~ co2_lag10 +
      ↪ volcano_linear + PDQ(D = 1)),
    arima_co2_3_nino = ARIMA(actual_temp ~ co2_lag3 + el_nino + la_nina +
      ↪ PDQ(D = 1)),
  )

```

```

    arima_co2_3_nino_tsi = ARIMA(actual_temp ~ co2_lag3 + el_nino + la_nina +
      ↪ TSI + PDQ(D = 1)),
    arima_physics_robust_decay = ARIMA(log(actual_temp) ~ rf_co2 +
      ↪ volcano_forcing + enso_delay + PDQ(D = 1)),
    arima_physics_robust_decayenso_3 = ARIMA(log(actual_temp) ~ rf_co2 +
      ↪ volcano_forcing + enso_smooth_3 + PDQ(D = 1)),
    arima_lag_3enso = ARIMA(log(actual_temp) ~ co2_lag3 + ENSO + PDQ(D=1)),
    arima_physicsenso_3 = ARIMA(log(actual_temp) ~ rf_co2 + enso_smooth_3 +
      ↪ PDQ(D=1))
  )

arima_second_fc <- arima_second_fit |>
  forecast(new_data = cv_valid)

arima_second_fc |>
  accuracy(cv_valid) |>
  select(.model, RMSE, ME, MAE) |>
  group_by(.model) |>
  summarize(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
  arrange(RMSE) |>
  kable(digits = 4)

```

.model	RMSE	MAE	ME
arima_auto_10enso_6	0.1225	0.0947	-0.0047
arima_physics_robust_decayenso_3	0.1233	0.0946	0.0005
arima_forcing	0.1266	0.1005	-0.0005
arima_robustenso_6	0.1273	0.1018	-0.0004
arima_physicsenso_3	0.1304	0.1012	-0.0011
arima_lag_3enso	0.1322	0.1044	0.0084
arima_auto_10	0.1356	0.1083	-0.0048
arimaenso_log_co2	0.1358	0.1076	-0.0304
arima_physics_robust	0.1376	0.1065	0.0158
arima_physics_robust_decay	0.1379	0.1064	0.0142
arima_co2_10_volcano	0.1597	0.1268	-0.0218
arima_co2_3_nino_tsi	0.1598	0.1275	0.0563
arima_co2_3_nino	0.1608	0.1290	0.0624

Across multiple folds, the forcing models and models that smooth out the enso index and/or utilize a lagged CO2 concentration variable perform the best. All the results are fairly close

and would probably trade places over different subsets of the data so choosing the best one will require consideration of other factors

```

arima_second_fit |>
  glance() |>
  select(.model, AICc, AIC, BIC) |>
  group_by(.model) |>
  summarize(across(.cols = c(AICc, AIC, BIC), .fns = mean)) |>
  arrange(AICc) |>
  kable(align = "c")

```

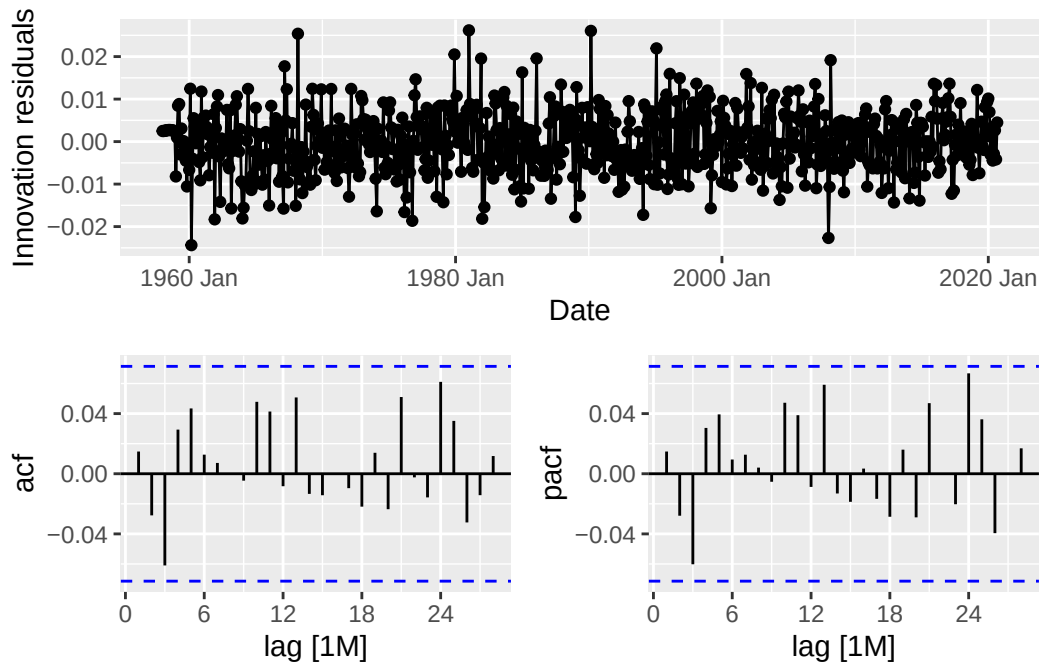
.model	AICc	AIC	BIC
arima_physics_robust_decayenso_3	-4299.9720	-4300.3234	-4256.9483
arima_auto_10	-4272.7611	-4272.9678	-4240.2055
arima_physicsenso_3	-4271.2614	-4271.5471	-4232.5938
arimaenso_log_co2	-4260.9085	-4261.1570	-4225.8582
arima_lag_3enso	-4257.9412	-4258.1951	-4221.9284
arima_auto_10enso_6	-4211.9089	-4212.1762	-4175.0569
arima_physics_robust	-4196.0189	-4196.3555	-4153.8237
arima_physics_robust_decay	-4195.2754	-4195.6120	-4153.0802
arima_robustenso_6	-4191.8260	-4192.1806	-4148.8485
arima_forcing	-4177.1452	-4177.4457	-4137.6491
arima_co2_10_volcano	-4140.9632	-4141.2000	-4105.7470
arima_co2_3_nino_tsi	-929.6644	-930.1066	-881.4664
arima_co2_3_nino	-917.6065	-917.9743	-873.7559

The top model by AICc was second best metric wise. It's also clear from the bottom models that a log transformation of the response variable drastically improves the models

```

arima_second_fit |>
  select(arima_physics_robust_decayenso_3) |>
  tail(n = 1) |>
  ggtime::gg_tsresiduals(plot_type = "partial")

```



```

arima_second_fit |>
  select(arima_physics_robust_decay_enso_3) |>
  tail(n = 1) |>
  augment() |>
  features(.innov, ljung_box, lag = 24) |>
  kable(digits = 3, align = "c")

```

.id	.model	lb_stat	lb_pvalue
5	arima_physics_robust_decay_enso_3	17.372	0.832

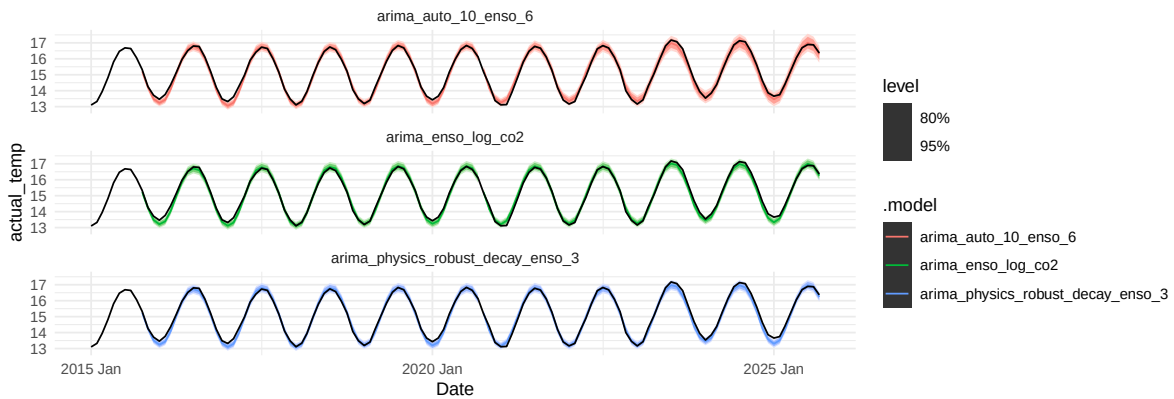
No autocorrelation spikes in the residuals and the Ljung-Box test confirms that the residuals resemble white noise for the `arima_physics_robust_decay_enso_3` model. Most other models have a spike at lag 3 or lag 24. The only other top model without any autocorrelation is `arima_physics_enso_3` so a 3 month smoothing of the ENSO index seems to take care of it

```

arima_second_fc |>
  filter(.id %in% c(4, 5), .model %in% c("arima_physics_robust_decay_enso_3",
    ↪ "arima_enso_log_co2", "arima_auto_10_enso_6")) |>
  autoplot() +
  autolayer(predictor_modeling |> filter(year(Date) >= 2015), actual_temp) +

```

```
facet_wrap(~ .model, ncol = 1) +
theme_minimal()
```



Retrain the top models on the same subset of data and compare

```
final_fit <- cv_trn |>
  model(
    ets_auto = ETS(actual_temp),
    ets_additive = ETS(actual_temp ~ error("A") + trend("A") + season("A")),
    ets_box_auto = ETS(box_cox(actual_temp, lambda = 1.5)),
    tslm_no_trend = TSLM(box_cox(actual_temp, lambda = 1.5) ~ season() +
      ↪ enso_smooth_12 + co2_lag10),
    tslm_volcanic = TSLM(box_cox(actual_temp, lambda = 1.5) ~ season() +
      ↪ enso_smooth_12 + co2_lag10 + volcano_forcing),
    tslm_enso_smooth_lag = TSLM(box_cox(actual_temp, lambda = 1.5) ~ trend()
      ↪ + season() + enso_smooth_6 + co2_lag10),
    arima_auto_10 = ARIMA(log(actual_temp) ~ ENSO + co2_lag10 + PDQ(D = 1)),
    arima_physics_robust_decay_enso_3 = ARIMA(log(actual_temp) ~ rf_co2 +
      ↪ volcano_forcing + enso_smooth_3 + PDQ(D = 1)),
    arima_physics_enso_3 = ARIMA(log(actual_temp) ~ rf_co2 + enso_smooth_3 +
      ↪ PDQ(D = 1)),
    arima_enso_log_co2 = ARIMA(log(actual_temp) ~ ENSO + co2_ppm + PDQ(D =
      ↪ 1)),
  )

final_fc <- final_fit |>
  forecast(new_data = cv_valid)

final_fc |>
```

```

accuracy(cv_valid) |>
select(.model, ME:MAPE) |>
group_by(.model) |>
summarize(across(.cols = c(RMSE, MAE, ME), .fns = mean)) |>
arrange(RMSE) |>
kable(digits = 4)

```

.model	RMSE	MAE	ME
tslm_no_trend	0.1183	0.0927	-0.0032
tslm_volcanic	0.1191	0.0936	0.0013
tslm_enso_smooth_lag	0.1217	0.0950	-0.0063
arima_physics_robust_decay_enso_3	0.1233	0.0946	0.0005
arima_physics_enso_3	0.1304	0.1012	-0.0011
arima_auto_10	0.1356	0.1083	-0.0048
arima_enso_log_co2	0.1358	0.1076	-0.0304
ets_additive	0.1569	0.1272	0.0397
ets_box_auto	0.1621	0.1331	0.0342
ets_auto	0.1656	0.1348	0.0604

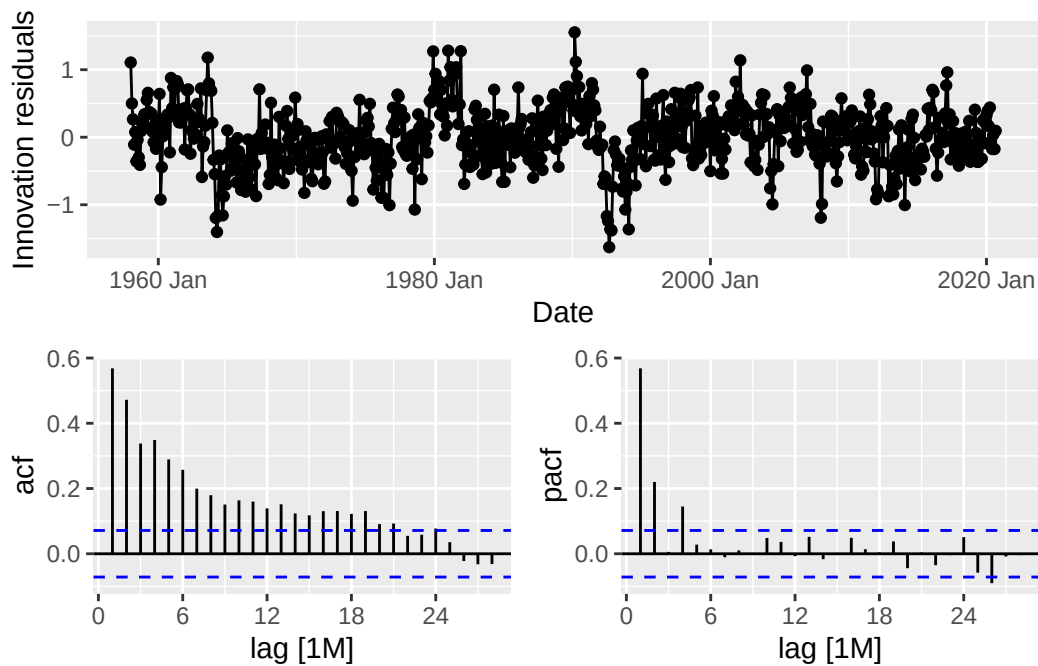
```

final_fit |>
select(tslm_no_trend) |>
tail(n = 1) |>
ggtime::gg_tsresiduals(plot_type = "partial")

```

Registered S3 methods overwritten by 'ggtime':

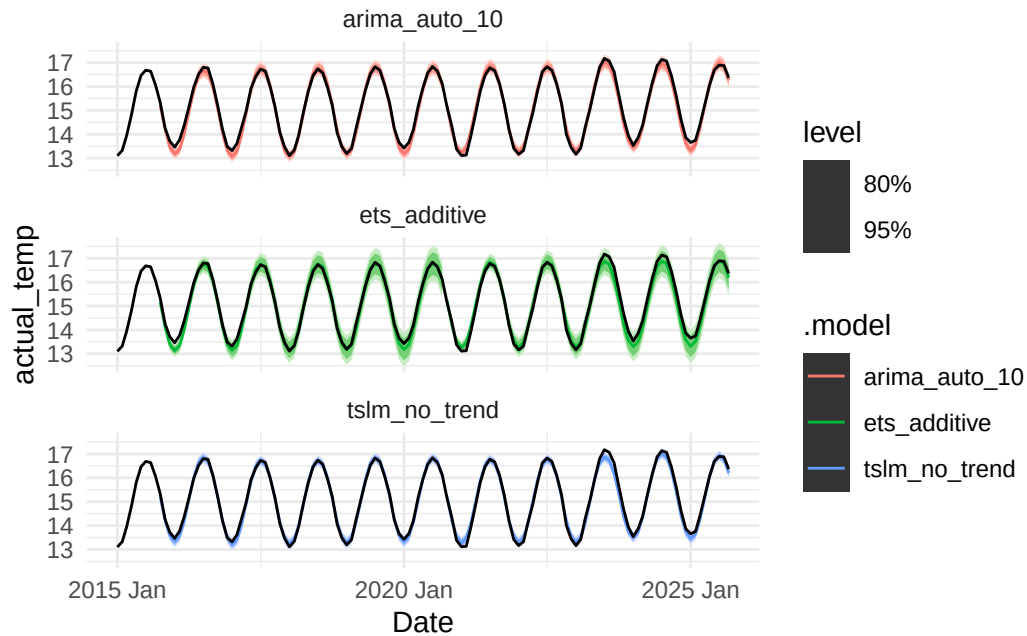
method	from
+.gg_tsensemble	feasts
autolayer.tbl_ts	fabletools
autoplot.dcmp_ts	fabletools
autoplot.tbl_ts	fabletools
grid.draw.gg_tsensemble	feasts
print.gg_tsensemble	feasts



```
final_fit |>
  select(arima_auto_10) |>
  augment() |>
  features(.innov, ljung_box, lag = 24)
```

```
# A tibble: 5 x 4
  .id .model      lb_stat lb_pvalue
<int> <chr>      <dbl>   <dbl>
1     1 arima_auto_10  19.0    0.749
2     2 arima_auto_10  45.6    0.00501
3     3 arima_auto_10  22.8    0.530
4     4 arima_auto_10  16.8    0.856
5     5 arima_auto_10  24.7    0.424
```

```
final_fc |>
  filter(.id %in% c(4, 5), .model %in% c("tslm_no_trend", "arima_auto_10",
    ↪ "ets_additive")) |>
  autoplot() +
  autolayer(predictor_modeling |> filter(year(Date) >= 2015), actual_temp) +
  facet_wrap(~ .model, ncol = 1) +
  theme_minimal()
```



```
current_baseline <- 3400921

model_data <- predictor_modeling |>
  mutate(
    lag_emissions = lag(aggregate_emissions, 12),
    enso_lag = lag(enso_smooth_12, 4)
  )

temp_model <- model_data |>
  filter(year(Date) > 1957) |>
  model(
    temp_arima = ARIMA(log(actual_temp) ~ 0 + co2_lag10 + enso_smooth_6 +
      ↪ pdq(3, 0, 2) + PDQ(0, 1, 1))
  )

# for the enso index, the average across the whole data set is 0.0470

low_enso <- predictor_modeling |>
  select(ENSO) |>
  slice_tail(n = 600)
# 0.0158

high_enso <- predictor_modeling |>
```

```

select(ENSO) |>
slice_head(n = 600)

# 0.0997

normal_enso <- predictor_modeling |>
  select(ENSO) |>
  slice(650:1249)

# 0.0469
#
# Then would need smoothed versions of these

co2_model <- model_data |>
  model(
    co2_arima = ARIMA(co2_ppm ~ lag_emissions + enso_lag + pdq(1, 1, 1) +
      ↪ PDQ(0, 1, 1)),
  )

# Model uses a 1 year lag on emissions and enso index is a 12 month smooth
↪ and 4 month lag

```

“noticed a spike at 12 and 24 for arima_auto_10 and auto was choosing ARIMA(3, 0, 2)(1, 1, 0) with enso_smooth_6 or ARIMA(2, 0, 2)(1, 1, 0) with enso_smooth_3. So added a sma term and that got rid of the spikes and the sar term showed as not significant so removed that. AICc was slightly higher using enso_smooth_6”

“Arima_auto_10_6 has lower AICc than all and residuals are white noise”

```

scenario_forecasting <- function(growth_rate, years, enso_level) {

  most_recent_observation <- predictor_modeling |> tail(1)
  baseline_aggregate <- most_recent_observation$aggregate_emissions

  annual_rate <- (1 + growth_rate/100)

  year_vector <- 1:years

  forecast <- current_baseline * (annual_rate ** year_vector)

  month_sequence <- rep(forecast, each = 12)
}

```

```

new_scenarios <- scenarios(
  enso_low = new_data(predictor_modeling, years * 12) |>
    mutate(
      ENSO = low_enso$ENSO,
    ),
  enso_norm = new_data(predictor_modeling, years * 12) |>
    mutate(
      ENSO = normal_enso$ENSO,
    ),
  enso_high = new_data(predictor_modeling, years * 12) |>
    mutate(
      ENSO = high_enso$ENSO,
    ),
  emissions = new_data(predictor_modeling, years * 12) |>
    mutate(
      total_co2 = month_sequence,
      aggregate_emissions = baseline_aggregate + cumsum(total_co2 / 1e6)
    )
)

co2_modeling_data <- new_scenarios[[enso_level]] |>
  left_join(new_scenarios$emissions, by = "Date") |>
  bind_rows(predictor_modeling) |>
  mutate(
    lag_emissions = lag(aggregate_emissions, 12),
    enso_smooth_3 = slider::slide_dbl(ENSO, mean, .before = 2),
    enso_smooth_6 = slider::slide_dbl(ENSO, mean, .before = 5),
    enso_smooth_12 = slider::slide_dbl(ENSO, mean, .before = 11),
    enso_lag = lag(enso_smooth_12, 4)
  )|>
  filter(Date >= yearmonth("2025 Oct"))

co2_ppm_fc <- co2_model |>
  forecast(new_data = co2_modeling_data)

co2_modeling_data$co2_ppm <- co2_ppm_fc$.mean

temp_modeling_data <- predictor_modeling |>
  bind_rows(co2_modeling_data) |>
  mutate(
    co2_lag10 = lag(co2_ppm, 120),
  ) |>
  filter(Date >= yearmonth("2025 Oct"))

```

```

temperature_forecast <- temp_model |>
  forecast(new_data = temp_modeling_data)

return(temperature_forecast)
}

```

External Predictor Feature Engineering Steps

```

ice_core <- read_csv("data/raw_data/annual_co2_icecore.csv")

loa_df <- read_csv("data/raw_data/mauna_loa_co2.csv")

ice_core |>
  filter(year(`C02 Date`) > 1849, year(`C02 Date`) <= 1955) |>
  add_row(
    `C02 Date` = as.Date("1956-01-01"),
    `C02 ppm` = 313
  ) |>
  add_row(
    `C02 Date` = as.Date("1957-01-01"),
    `C02 ppm` = 313
  )

co2_ts <- ice_core |>
  mutate(
    Year = year(`C02 Date`),
    Month = list(1:12)
  ) |>
  unnest(Month) |>
  mutate(
    Date = as.Date(glue::glue("{Year}-{Month}-01")),
    Date = yearmonth(Date)
  ) |>
  select(c(Date, `C02 PPM`)) |>
  filter(year(Date) > 1849, year(Date) < 1958) |>
  as_tsibble() |>
  fill_gaps(.start = yearmonth("1850-01"), .end = yearmonth("1958-02")) |>
  fill(`C02 PPM`, .direction = "down")

```

```

loa_ts <- loa_df |>
  mutate(
    Date = as.Date(glue::glue("{year}-{month}-01")),
    Date = yearmonth(Date)
  ) |>
  rename(`CO2 PPM` = average) |>
  select(Date, `CO2 PPM`) |>
  as_tibble()

co2_ppm <- bind_rows(co2_ts, loa_ts)

```

The annual data for ice core co2 was cut between 1850 and February 1958, then fill gaps was used to create NA rows for monthly data in between years, then the NA values were filled in with the most recent known observation. This data was then joined with the known observations from Mauna Loa

```

# Annual CO2

annual_co2 <- read_csv("data/raw_data/annual_co2_emissions.csv")

co2_land_use <- read_csv("data/raw_data/annual_co2_land_use_emissions.csv")

# Monthly CO2

monthly_co2 <-
  ↪ read_rds("data/raw_data/global_monthly_emissions_1970_2024.rds")

monthly_co2_bio <-
  ↪ read_rds("data/raw_data/global_monthly_emissions_landuse_1970_2024.rds")

month_co2 <- monthly_co2 |>
  select(c(Year:Dec)) |>
  group_by(Year) |>
  summarize(across(.cols = c(Jan:Dec), .fns = sum)) |>
  pivot_longer(cols = c(Jan:Dec), names_to = 'Month', values_to =
    ↪ "CO2_Emissions") |>
  mutate(
    date_string = paste(Year, Month, "01", sep = "-"),
    Date = yearmonth(date_string)
  ) |>

```

```

select(c(Date, CO2_Emissions))

month_co2_bio <- monthly_co2_bio |>
  select(c(Year:Dec)) |>
  group_by(Year) |>
  summarize(across(.cols = c(Jan:Dec), .fns = sum)) |>
  pivot_longer(cols = c(Jan:Dec), names_to = 'Month', values_to =
    ↪ "CO2_Bio_Emissions") |>
  mutate(
    date_string = paste(Year, Month, "01", sep = "-"),
    Date = yearmonth(date_string)
  ) |>
  select(c(Date, CO2_Bio_Emissions))

combined_monthly <- month_co2 |>
  left_join(month_co2_bio)

final_monthly <- combined_monthly |>
  mutate(Total_CO2 = CO2_Bio_Emissions + CO2_Emissions) |>
  select(c(Date, Total_CO2))

annual_convert <- annual_co2 |>
  filter(Year < 1970) |>
  uncount(12, .id = "Month") |>
  mutate(
    Date = make_yearmonth(year = Year, month = Month),
    monthly_co2 = (World * 3664) / 12
  ) |>
  select(Date, monthly_co2) |>
  as_tsibble(index = Date)

annual_bio_convert <- co2_land_use |>
  filter(Year <= 1969) |>
  uncount(12, .id = "Month") |>
  mutate(
    Date = make_yearmonth(year = Year, month = Month),
    monthly_bio_co2 = (Average * 3664) / 12
  ) |>
  select(Date, monthly_bio_co2) |>
  as_tsibble(index = Date)

```

```
combined_monthly_convert <- annual_convert |>
  left_join(annual_bio_convert)

earlier_co2_data <- combined_monthly_convert |>
  mutate(Total_CO2 = (monthly_bio_co2 + monthly_co2)) |>
  select(c(Date, Total_CO2))
```

Monthly CO2 emission estimates from 1970 - 2024 were combined with annual emissions dating back to 1850. The annual estimates were in million tons of carbon, while the monthly estimates were in gigagrams of CO2, which is equal to 1000 metric tons. First, to convert the annual estimates to gigagrams, the values need to be multiplied by 3.664 to convert carbon to CO2 weight, then multiplied by 1000 to convert to gigagrams

```
global_temps <- read_csv("data/converted_global_temp.csv")

tsi <- read_rds("data/raw_data/monthly_tsi.rds")

ch4 <- read_rds("data/raw_data/annual_ch4.rds")

full_ch4 <- ch4 |>
  filter(year(`CH4 Date`) >= 1849) |>
  mutate(Date = yearmonth(`CH4 Date`)) |>
  rename(ch4 = `CH4 Value`) |>
  select(Date, ch4) |>
  as_tsibble() |>
  fill_gaps(.start = yearmonth("1850-01"), .end = yearmonth('2025-09')) |>
  fill(ch4, .direction = "down")

full_tsi <- tsi |>
  mutate(Date = yearmonth(time)) |>
  select(Date, TSI) |>
  as_tsibble() |>
  fill_gaps(.start = yearmonth("1850-01"), .end = yearmonth('2025-09')) |>
  fill(TSI, .direction = "updown")

co2_ts <- co2_ppm |>
  mutate(Date = yearmonth(Date)) |>
  rename(co2_ppm = `CO2 PPM`) |>
  select(Date, co2_ppm) |>
  as_tsibble()
```

```

full_reg <- global_temps |>
  select(Date, actual_temp) |>
  mutate(Date = yearmonth(Date)) |>
  as_tsibble() |>
  left_join(earlier_co2_data, by = 'Date') |>
  left_join(full_ch4, by = "Date") |>
  left_join(full_tsi, by = "Date") |>
  left_join(co2_ts, by = "Date")

```

The annual data for ch4 was cutoff from 1850. Fill gaps can create NA rows for monthly data in between years, then the NA's can be fill in with the most recent known observation.

The TSI data extends from 1874 to 2023 so a similar process can be used to extend it from 1850 to Sept 2025.

```

new_df <- read_rds("data/raw_data/enso_index.rds")

row_index <- which(new_df$Year == 2024)

if (length(row_index) == 1) {
  new_df[row_index, "MAR"] <- 1.13
  new_df[row_index, "APR"] <- 0.77
  new_df[row_index, "MAY"] <- 0.23
  new_df[row_index, "JUN"] <- 0.17
  new_df[row_index, "JUL"] <- 0.04
  new_df[row_index, "AUG"] <- -0.12
  new_df[row_index, "SEP"] <- -0.26
  new_df[row_index, "OCT"] <- -0.27
  new_df[row_index, "NOV"] <- -0.25
  new_df[row_index, "DEC"] <- -0.60
}

new_df_long <- new_df |>
  pivot_longer(
    cols = c(JAN:DEC),
    names_to = "Month",
    values_to = "ENSO"
  ) |>
  mutate(
    Date_String = glue::glue("{Year}-{Month}-01"),
    Date = yearmonth(as.Date(Date_String, format = "%Y-%b-%d"))
  )

```

```

) |>
select(Date, ENSO) |>
as_tsibble()

new_df_long <- new_df_long |>
select(-enso_smooth) |>
mutate(
  enso_smooth_12 = slider::slide_dbl(ENSO, mean, .before = 11, complete =
    ↪ TRUE),
  enso_smooth_6 = slider::slide_dbl(ENSO, mean, .before = 5, complete =
    ↪ TRUE),
  enso_smooth_3 = slider::slide_dbl(ENSO, mean, .before = 2, complete =
    ↪ TRUE),
  enso_delay = lag(enso_smooth_3, 4),
  rf_co2 = 5.35 * log(co2_ppm / 278)
)

```

The enso index data from the Ensemble ONI websites most recent observation is February of 2024. NOAA has the most up to date values so a vector of up to date values was used to fill in the rest of the year. Then 3, 6 and 12 month smoothed versions of the index were created to reduce the effect of noise. Then a delayed version of the index was created by using a 4-month lag on the 3 month smoothed index.

The radiative forcing metric of CO2 was evaluated by taking the log transformation of the CO2 concentration divided by the pre-industrial baseline then multiplied by 5.35. This is a simplified version of the more complex IPCC formula.

```

library(zoo)

# CO2 data

mauna_loa <- read_csv("data/raw_data/mauna_loa_co2.csv")
icecore <- read_csv("data/raw_data/annual_co2_icecore.csv")

# Annual estimates
n2o <- read_csv("data/raw_data/n2o_annual.csv")
ch4 <- read_csv("data/raw_data/ch4_annual.csv")

# Monthly estimates
mauna_n2o <- read_csv("data/raw_data/n2o_monthly.csv")
mauna_ch4 <- read_csv("data/raw_data/ch4_monthly.csv")

```

```

# Convert to tsibble
loa_ts <- mauna_loa |>
  mutate(
    Date = make_yearmonth(year = year, month = month)
  ) |>
  rename(co2_ppm = average) |>
  select(Date, co2_ppm) |>
  as_tsibble()

# Average seasonal component for known observations
seasonal_trend <- mauna_loa |>
  mutate(
    Date = make_yearmonth(year = year, month = month)
  ) |>
  select(Date, average) |>
  as_tsibble() |>
  model(STL()) |>
  components() |>
  as_tibble() |>
  group_by(month(Date)) |>
  summarise(co2_season = mean(season_year)) |>
  rename(month = `month(Date)`)

# Regress and interpolate annual estimates up until most recent known monthly
  ↪ observation
full_ts <- icecore |>
  mutate(
    Date = yearmonth(`CO2 Date`)
  ) |>
  rename(co2 = `CO2 PPM`) |>
  select(Date, co2) |>
  complete(Date = seq(min(Date), max(Date), by = 1)) |>
  filter(between(year(Date), 1850, 1957)) |>
  complete(Date = seq(min(Date), yearmonth("1958-02"), by = 1)) |>
  as_tsibble() |>
  mutate(
    co2_ppm = na.spline(co2)
  ) |>
  mutate(month = month(Date)) |>
  left_join(seasonal_trend, by = "month") |>
  mutate(
    scale_factor = co2_ppm / mean(mauna_loa$average),
    final_co2 = co2_ppm + (co2_season * scale_factor)
  )

```

```

) |>
select(Date, final_co2) |>
rename(co2_ppm = final_co2) |>
bind_rows(loa_ts)

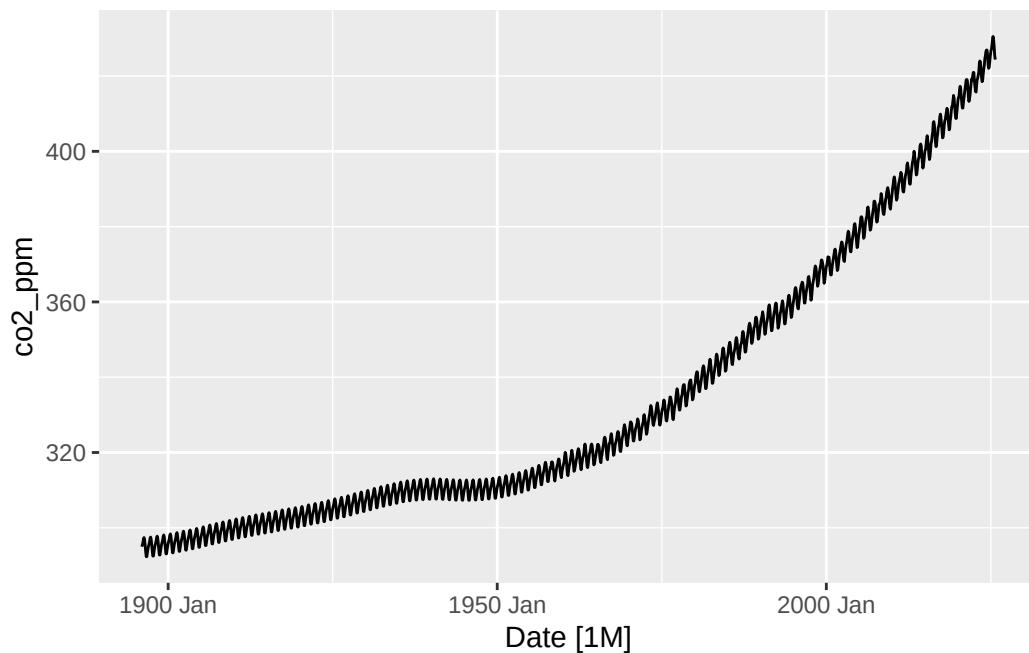
```

The `zoo` library can be used to perform a spline regression on the annual estimates we replicated across monthly values to even out the step curve. Then the average seasonal component can be estimated from the true monthly observations, multiplied by a scale factor, then added back to the corresponding interpolated observation. The results can be see below

```

predictor_modeling |>
  autoplot(co2_ppm)

```



The trend remains the same but the seasonal pattern now looks similar to the post-1957 observations

```

new_df <- new_df |>
  select(-enso_smooth) |>
  mutate(
    enso_smooth_12 = slider::slide_dbl(ENSO, mean, .before = 11, complete =
      ↪ TRUE),

```

```

    enso_smooth_6 = slider::slide_dbl(ENSO, mean, .before = 5, complete =
      ↪ TRUE),
    enso_smooth_3 = slider::slide_dbl(ENSO, mean, .before = 2, complete =
      ↪ TRUE),
    enso_delay = lag(enso_smooth_3, 4),
    rf_co2 = 5.35 * log(co2_ppm / 278)
  )

df <-
  ↪ read_tsv("C:/Users/tkbar/Downloads/volcano-events-2025-12-03_09-46-47_-0800.tsv")

volcano <- df |>
  select(Year, Mo, Name, VEI) |>
  drop_na() |>
  filter(VEI > 0) |>
  mutate(
    Date = make_yearmonth(year = Year, month = Mo),

  ) |>
  select(Date, VEI)

df |>
  filter(Year == 2022)

volcano_clean <- volcano %>%
  filter(VEI >= 4)

calculate_decay <- function(current_date, eruption_table, decay_months = 36,
  ↪ tau = 12) {

  curr_d <- as.Date(current_date)
  erup_d <- as.Date(eruption_table$Date)

  window_start <- curr_d - months(decay_months)

  active_indices <- which(erup_d <= curr_d & erup_d > window_start)

  if (length(active_indices) == 0) {
    return(0)
  }
}

```

```

time_span <- lubridate::interval(erup_d[active_indices], curr_d)
months_elapsed <- time_span %/% months(1)

strengths <- exp(-months_elapsed / tau)

return(sum(strengths))
}

climate_tsibble_updated <- lagged %>%
  mutate(
    volcano_forcing = map_dbl(Date, calculate_decay, eruption_table =
      ↪ volcano_clean)
  )

```

The volcanic events with a lower VEI than 4 are filtered out as their impact on the climate is most likely too minimal to significantly impact the global climate. A dummy variable is created for the presence of the event, then an exponential decay formula extending over the course of 3 years was applied. Overlapping events were summed